SATGUARD: A LOW-CODE SATELLITE MONITORING AND ALERT MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

KIRITHIKA S
(241401041)

in partial fulfilment of the End Semester Examination for the course
CB23F35 –
LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF TECHNOLOGY
COMPUTER SCIENCE AND BUSINESS SYSTEMS



RAJALAKSHMI ENGINEERING COLLEGE
(An Autonomous Institution, Affiliated to Anna University, Chennai)
Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "**SATGUARD: A LOW-CODE SATELLITE MONITORING AND ALERT MANAGEMENT SYSTEM**" is the bonafide work of **KIRITHIKA S (241401041)**, who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

Dr. K. R. Sowmia, Ph.D.
HEAD OF THE DEPARTMENT
Department of Computer Science
and Business Systems
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Mr. M. L. Alphin Ezhil Manuel
COURSE FACULTY
Assistant Professor (SG)
Department of Computer Science
and Business Systems
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, **KIRITHIKA S (241401041)**, hereby declare that the project report titled "**SATGUARD: A LOW-CODE SATELLITE MONITORING AND ALERT MANAGEMENT SYSTEM**", submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Chennai

Date:

SIGNATURE OF THE STUDENT
KIRITHIKA S
(241401041)

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman **Mr. S. Meganathan, B.E., F.L.E.,** Chairperson **Dr. Thangam Meganathan, Ph.D.,** and Vice-Chairman **Mr. Abhay Shankar Meganathan, B.E., M.S.,** for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, **Dr. S. N. Murugesan, M.E., Ph.D.,** Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank **Dr. K. R. Sowmia, Ph.D.,** Head of the Department, Department of Computer Science and Business Systems, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, **Mr. M. L. Alphin Ezhil Manuel, Assistant Professor (SG),** for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Creator Education Programme for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

KIRITHIKA S

ABSTRACT

Satellite monitoring involves tracking satellite information, monitoring telemetry parameters, identifying abnormal conditions, and managing alerts and maintenance activities. Traditional manual monitoring and spreadsheet-based approaches can make it difficult to maintain organised records, identify critical conditions promptly, and track unresolved alerts efficiently. These limitations highlight the need for a centralised system that simplifies satellite data management and improves the reliability of monitoring and alert handling.

SATGUARD is a low-code satellite monitoring and alert management application developed using Zoho Creator. The application includes modules for satellite registration, telemetry logging, alert management, and maintenance tracking, along with a dashboard for summarising satellite status, open alerts, and maintenance tasks. Deluge scripting and automated workflows are used to generate alerts for communication failures, low battery voltage, high temperature, and low power levels. The system also prevents duplicate open alerts, records maintenance completion dates automatically, and sends email notifications when critical alerts are created. These features help organise monitoring activities and reduce repetitive manual work.

The application was tested using sample satellite records and telemetry inputs to verify field validation, automatic alert generation, duplicate-alert prevention, maintenance completion tracking, and critical alert notifications. The dashboard and reports provide a consolidated view of satellite status and operational issues. By combining data management, workflow automation, and alert tracking in a single low-code platform, SATGUARD demonstrates a practical approach to improving monitoring efficiency and simplifying maintenance coordination. The application is designed to support timely identification of potential issues and more systematic management of satellite monitoring activities.

Keywords: Low-Code Development; Zoho Creator; Deluge Scripting; Satellite Monitoring; Telemetry Analysis; Workflow Automation; Alert Management; Maintenance Tracking.

PROJECT AT A GLANCE

Attribute	Details
Project Title	SATGUARD – A Low-Code Platform for Satellite Telemetry Monitoring, Anomaly Detection, and Automated Alert Management
Course Code & Title	CB23F35 – Low-Code Application Development using Zoho Creator
Application Domain	Aerospace Technology / Satellite Monitoring and Maintenance Management
Student Name & Register No.	Kirithika S – 241401041
Department & Institution	Computer Science and Business Systems, Rajalakshmi Engineering College (An Autonomous Institution), Thandalam, Chennai – 602 105
Course Faculty	Mr. M. L. Alphin Ezhil Manuel, Assistant Professor (SG)
Zoho Creator Edition	Education
Application Name (Link Name)	SATGUARD
Published / Live URL	https://creatorapp.zohopublic.in/wrongterminal2026/satguard/page-perma/SATGUARD_DASHBOARD/msMuVeCwTEWWm7g6kWtDe4QHxXr907DN496nqOfsS7uQ07R5dgK5pjMd45XV6xfeypE4xHTfdTwybVgZ9BWt6HHM90aJ9Es7vAt5
Demo Credentials (per role)	Administrator, Monitoring Staff, Maintenance Staff
Number of Forms / Subforms	4 Forms – Satellite Registry, Telemetry Log, Alert Log, Maintenance Log
Number of Reports (by type)	List: 4 – Satellite Registry, Telemetry Log, Alert Log and Maintenance Log Reports
Number of Pages / Dashboards	1 Dashboard – SATGUARD Dashboard
Number of Workflows	Form workflows: 5 implemented workflow categories – communication failure alerts, telemetry threshold alerts, duplicate-alert prevention, maintenance completion date, and critical alert email notification
Deluge Functions / Custom Actions	Deluge scripts for telemetry validation, automatic alert generation, duplicate-alert prevention, and maintenance completion tracking

Attribute	Details
Integrations	Zoho CRM – critical alert details sent to the CRM Cases module
Roles & Profiles Defined	Administrator, Monitoring Staff, Maintenance Staff
AI / Analytics Features	Rule-based threshold monitoring with dashboard and reports; no machine-learning prediction used
Mobile & Portal Access	Browser / Zoho Creator platform-supported access; no separate customer portal
Environments Used	Zoho Creator development environment → Stage → Production
Date of Production Deployment	20-10-2026

TABLE OF CONTENTS

BONAFIDE CERTIFICATE....i
DECLARATION....ii
ACKNOWLEDGEMENT....iii
ABSTRACT....iv
PROJECT AT A GLANCE....v
TABLE OF CONTENTS....vii
LIST OF TABLES....ix
LIST OF FIGURES....x
LIST OF ABBREVIATIONS....xi
CHAPTER 1: INTRODUCTION....1
1.1 Background....1
1.2 Problem Statement....1
1.3 Objectives....2
1.4 Scope of the Project....3
1.5 Justification for a Low-Code Approach....4
1.6 Organisation of the Report....4
CHAPTER 2: LITERATURE SURVEY AND SYSTEM STUDY....6
2.1 Study of the Existing System....6
2.2 Limitations of the Existing System....7
2.3 Low-Code versus Traditional Development....7
2.4 Literature Survey....8
2.5 Emerging Trends in Low-Code Development....10
2.6 Proposed System....10
CHAPTER 3: REQUIREMENT ANALYSIS....12
3.1 Stakeholders and User Roles....12
3.2 Functional Requirements....12
3.3 Non-Functional Requirements....13
3.4 Hardware and Software Requirements....14
3.5 Use Case Model....14
CHAPTER 4: SYSTEM DESIGN....16
4.1 System Architecture....16
4.2 Data Model....17
4.3 Form Design....18
4.4 Process and Workflow Design....19
4.4.1 Telemetry Validation Process....19
4.4.2 Automated Alert Generation....20
4.4.3 Maintenance Completion Workflow....21
4.5 User Interface and Navigation Design....21
4.6 Access Control Design....22
CHAPTER 5: IMPLEMENTATION....24
5.1 Application Setup....24

5.2 Forms and Data Capture....24
5.2.1 Satellite Registry....25
5.2.2 Telemetry Log....25
5.2.3 Alert Log....26
5.2.4 Maintenance Log....27
5.3 Reports....27
5.4 Pages and Dashboards....29
5.5 Deluge Scripting....30
5.6 Workflows and Automation....33
5.7 Integrations....34
5.8 User Management and Role-Based Access....35
5.9 AI and Analytics Features....36
5.10 Pro-Code Extension....37
5.11 Mobile and Portal Access....37
CHAPTER 6: TESTING AND DEBUGGING....39
6.1 Testing Strategy....39
6.2 Test Cases....39
6.3 Debugging Deluge Scripts....40
6.4 Defect Log....41
6.5 User Acceptance Testing....42
CHAPTER 7: DEPLOYMENT, SECURITY AND GOVERNANCE....43
7.1 Environments and Deployment Strategy....43
7.2 Performance Optimisation....44
7.3 Security Features....45
7.4 Governance and Administration....46
CHAPTER 8: RESULTS AND DISCUSSION....48
8.1 Application Output....48
8.2 Achievement of Objectives....50
8.3 Comparison with the Existing System....50
8.4 Limitations of the Low-Code Approach....51
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum....52
CHAPTER 9: CONCLUSION AND FUTURE ENHANCEMENTS....53
9.1 Conclusion....53
9.2 Future Enhancements....53
9.3 Learning Outcomes and Career Relevance....54
REFERENCES....55
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST....56
APPENDIX B – COURSE OUTCOME MAPPING....59
APPENDIX C – COMPLETE DELUGE SOURCE CODE....63
APPENDIX D – USER MANUAL....66

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application....8
Table 2.2: Summary of Literature Survey....9
Table 3.1: Stakeholders and User Roles....12
Table 3.2: Functional Requirements....12
Table 3.3: Non-Functional Requirements....13
Table 3.4: Hardware and Software Requirements....14
Table 3.5: Use Case Description – Register Satellite....15
Table 4.1: Entities and Relationships....18
Table 4.2: Field Specification – Satellite Registry....19
Table 4.3: Role–Permission Matrix....23
Table 5.1: Reports Implemented....27
Table 5.2: Deluge Scripts Implemented....30
Table 5.3: Workflows Implemented....33
Table 5.4: Integrations Implemented....34
Table 6.1: Test Cases....39
Table 6.2: Defect Log....41
Table 6.3: User Acceptance Feedback....42
Table 7.1: Deployment Checklist....43
Table 7.2: Performance Optimisation Results....44
Table 7.3: Security Controls Implemented....45
Table 8.1: Mapping of Objectives to Outcomes....50
Table 8.2: Existing System versus Proposed Application....50
Table A.1: Feature Coverage Checklist....56
Table B.1: Mapping of Course Outcomes to Report Chapters....59
Table B.2: Mapping of Syllabus Units to the Project....60
Table B.3: Mapping of Laboratory Experiments to the Project....60
Table B.4: CO–PO–PSO Matrix of the Course CB23F35....61
Table D.1: Common Messages and Actions....67

LIST OF FIGURES

Figure 2.1: As-Is Process Flow of the Existing System.....7
Figure 3.1: Use Case Diagram of the Application.....15
Figure 4.1: System Architecture of the Application.....17
Figure 4.2: Entity–Relationship Diagram.....18
Figure 4.3: Workflow / Process Flow for Telemetry Validation.....20
Figure 4.4: Deluge Script for Automated Alert Generation.....21
Figure 4.5: Deluge Script for Maintenance Completion Date.....21
Figure 4.6: Navigation Structure and Wireframe of Dashboard.....22
Figure 5.1: Application Home and Navigation Sections.....24
Figure 5.2: Satellite Registry Form in Live Mode.....25
Figure 5.3: Telemetry Log Form in Live Mode.....26
Figure 5.4: Alert Log Form in Live Mode.....26
Figure 5.5: Maintenance Log Form in Live Mode.....27
Figure 5.6: Satellite Registry Report.....28
Figure 5.7: Telemetry Log Report.....28
Figure 5.8: Alert Log Report.....28
Figure 5.9: Maintenance Log Report.....29
Figure 5.10: SATGUARD Dashboard.....29
Figure 5.11: Critical Alert Notification Workflow.....34
Figure 5.12: Workflow Configuration.....34
Figure 5.13: Zoho CRM Connection Setup.....35
Figure 5.14: Roles.....36
Figure 5.15: Permissions.....36
Figure 5.16: Dashboard Analytics Components.....37
Figure 5.17: Application on Mobile.....38
Figure 6.1: Critical Alert Email Notification.....41
Figure 7.1: Environments.....43
Figure 7.2: Administration Settings.....47
Figure 8.1: Scenario Step 1.....48
Figure 8.2: Scenario Step 2.....49
Figure 8.3: Scenario Step 3.....50

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CO	Course Outcome
CRM	Customer Relationship Management
CRUD	Create, Read, Update, Delete
Deluge	Data Enriched Language for the Universal Grid Environment
ER	Entity–Relationship
FR	Functional Requirement
JSON	JavaScript Object Notation
LCAP	Low-Code Application Platform
NFR	Non-Functional Requirement
OAuth	Open Authorization
PO	Programme Outcome
PSO	Programme Specific Outcome
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDLC	Software Development Life Cycle
SSO	Single Sign-On
UAT	User Acceptance Testing
UI	User Interface
Zia	Zoho’s AI assistant

CHAPTER 1
INTRODUCTION

1.1 Background

Satellite technology plays an important role in communication, navigation, weather forecasting, remote sensing, scientific research, and space exploration. Satellites continuously operate in space and require regular monitoring to ensure that their essential parameters remain within acceptable limits. Monitoring information such as battery voltage, temperature, power level, communication status, and operational condition helps identify potential issues and supports timely maintenance decisions.

Managing satellite information manually or through separate spreadsheets can make it difficult to maintain organised records, monitor telemetry readings, identify abnormal conditions, and track unresolved alerts. Such approaches may also increase repetitive work and delay the identification of important operational issues. Therefore, a centralised application that combines satellite registration, telemetry logging, alert management, and maintenance tracking can improve the organisation and accessibility of monitoring information.

SATGUARD is a low-code satellite monitoring and alert management application developed using Zoho Creator. The application provides structured forms for maintaining satellite records, recording telemetry readings, managing alerts, and tracking maintenance activities. It uses Deluge scripting and automated workflows to identify predefined abnormal conditions, create alerts, prevent duplicate open alerts, record maintenance completion dates, and send email notifications for critical alerts. A dashboard and reports provide a consolidated view of satellite status, open alerts, and maintenance activities. The project demonstrates how a low-code platform can simplify the development of an organised monitoring application while reducing repetitive manual tasks.

1.2 Problem Statement

Managing satellite information, telemetry readings, alerts, and maintenance activities through disconnected records or manual processes can make monitoring less efficient. Without a centralised application, users may find it

difficult to identify abnormal telemetry values, track communication failures, monitor unresolved alerts, and maintain consistent records of maintenance activities. Delayed identification of potential issues and repetitive manual data handling can affect the effectiveness of monitoring and decision-making.

The major problems addressed by SATGUARD are:

- **Fragmented information management:** Satellite details, telemetry readings, alerts, and maintenance records may be difficult to organise and access when maintained separately.
- **Delayed identification of abnormal conditions:** Manual monitoring can make it harder to identify low battery voltage, high temperature, low power levels, and communication failures promptly.
- **Inefficient alert and maintenance tracking:** Duplicate alerts, unresolved issues, and incomplete maintenance records can complicate the monitoring process and make it harder to track corrective actions.

SATGUARD addresses these problems by providing a centralised low-code application with automated alert generation, maintenance tracking, and dashboard-based reporting.

1.3 Objectives

The primary objective of SATGUARD is to develop a centralised low-code application for managing satellite records, monitoring telemetry parameters, generating alerts, and tracking maintenance activities.

The specific objectives are:

- To develop structured forms for registering satellites and maintaining telemetry, alert, and maintenance records.
- To automate the identification of predefined telemetry threshold violations and communication failures using Deluge scripting and workflows.
- To implement duplicate-alert prevention and automatically record maintenance completion dates.
- To provide a dashboard and reports for viewing satellite operational status, open alerts, and maintenance activities.

- To configure email notifications for critical alerts to support timely investigation.
- To test the application's data validation, alert generation, workflow execution, and reporting functionality using sample records.

1.4 Scope of the Project

The scope of SATGUARD is limited to the development of a low-code satellite monitoring and alert management application using Zoho Creator. The system focuses on organising satellite information, recording sample telemetry readings, identifying predefined abnormal conditions, managing alerts, and maintaining maintenance records.

In-scope activities:

- Registering satellites and maintaining mission and operational details.
- Recording telemetry parameters such as battery voltage, temperature, power level, and communication status.
- Generating alerts for configured conditions, including low battery voltage, high temperature, low power level, and communication failure.
- Preventing duplicate open alerts for the same satellite and alert type.
- Maintaining alert severity, timestamps, descriptions, and resolution status.
- Tracking maintenance tasks and automatically recording the completion date when a task is marked as completed.
- Sending email notifications when a critical alert is created.
- Displaying satellite and maintenance information through reports and a dashboard.
- Testing the implemented workflows using sample data.

Out-of-scope activities:

- Direct communication with satellites or ground stations.
- Receiving live telemetry directly from spacecraft hardware.
- Controlling satellite operations, onboard systems, or orbital manoeuvres.

- Performing operational collision avoidance or guaranteeing real-time orbital collision predictions.
- Implementing a validated spacecraft-grade anomaly detection model.
- Integrating external satellite telemetry APIs unless separately configured and tested.

The current application is intended as a low-code monitoring and record-management prototype. Its alerts depend on the telemetry values entered into the application and the thresholds configured in its workflows.

1.5 Justification for a Low-Code Approach

A low-code development approach was selected for SATGUARD because the project requires structured data entry, record management, automated workflows, reports, and a dashboard within a limited development period. Developing all these components from scratch would require additional effort for frontend development, backend programming, database operations, authentication, and reporting.

Zoho Creator provides a platform for building forms, reports, dashboards, and workflow automation with reduced manual coding. Its Deluge scripting capability supports application-specific logic, including telemetry threshold checks, alert creation, duplicate-alert prevention, and maintenance completion tracking. The platform also supports email notification functionality, allowing critical alerts to be communicated to designated recipients.

The low-code approach enables faster application development, easier modification of forms and workflows, and centralised management of records. It is therefore suitable for developing SATGUARD as an academic prototype that demonstrates the practical application of workflow automation and structured data management in satellite monitoring. However, the prototype does not replace validated spacecraft monitoring infrastructure or operational satellite-control systems.

1.6 Organisation of the Report

This report is organised into nine chapters. Chapter 1 introduces the project background, problem statement, objectives, scope, and justification for adopting a low-code development approach. Chapter 2 reviews the existing

system, related approaches, and relevant low-code application development concepts. Chapter 3 presents the system requirements, including functional and non-functional requirements. Chapter 4 describes the system design, architecture, data organisation, and workflow design. Chapter 5 explains the implementation of SATGUARD using Zoho Creator, including forms, reports, dashboard components, Deluge scripts, and automated workflows. Chapter 6 presents the testing and debugging procedures used to verify the implemented features. Chapter 7 discusses deployment, performance considerations, application security, and governance. Chapter 8 presents the results and discussion based on the implemented features and testing outcomes. Finally, Chapter 9 concludes the project and identifies possible future enhancements.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

Satellite monitoring requires the systematic management of satellite information, telemetry readings, operational alerts, and maintenance activities. In a conventional approach, these details may be recorded in spreadsheets, separate documents, or independent monitoring tools. Users may need to enter telemetry values manually, inspect the readings, identify abnormal conditions, and update alert and maintenance records separately.

The existing manual or spreadsheet-based process generally begins with maintaining satellite information. Telemetry readings such as battery voltage, temperature, power level, and communication status are then recorded. Users review the values against predefined limits and create alerts when an abnormal condition is identified. The alerts are investigated, maintenance activities are recorded, and the corresponding records are updated after corrective actions are completed.

When these activities are handled independently, maintaining consistent records and identifying unresolved issues can become difficult. Manual checking may also delay alert generation, while repeated data entry can lead to duplicate or incomplete records. A centralised system with automated rules can help organise these activities and reduce repetitive work.

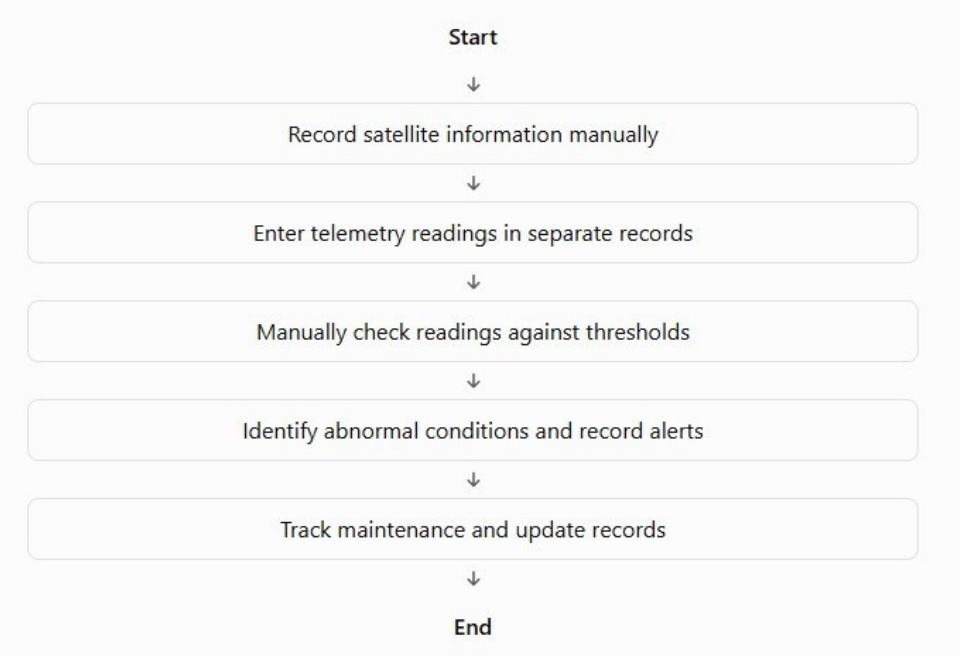


Figure 2.1: As-Is Process Flow of the Existing System

2.2 Limitations of the Existing System

The major limitations of the existing manual or spreadsheet-based approach are:

- **Fragmented data management:** Satellite details, telemetry records, alerts, and maintenance information may be stored separately, making it difficult to obtain a consolidated view of operational status.
- **Dependence on manual monitoring:** Users must repeatedly inspect telemetry readings and compare them against configured limits, which may delay the identification and recording of abnormal conditions.
- **Inefficient alert and maintenance tracking:** Repeated entries, delayed status updates, and incomplete maintenance records can complicate the monitoring process and reduce the efficiency of follow-up activities.

These limitations motivate the development of SATGUARD, which combines structured records, automated alert generation, maintenance tracking, and dashboard-based reporting in a single application.

2.3 Low-Code versus Traditional Development

Traditional development involves designing and programming application components using languages, frameworks, databases, and deployment tools.

This approach provides extensive customisation and control but requires more development effort and technical expertise. In contrast, low-code development platforms provide visual tools for creating forms, reports, workflows, and application interfaces, while allowing scripts to implement specialised business logic.

For SATGUARD, Zoho Creator was selected to reduce the effort required to develop the data-entry forms, reports, dashboard, and automation workflows. Deluge scripting was used where application-specific logic was necessary. The following table compares both approaches in the context of the proposed application.

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	Several weeks of coding and testing	Faster development using visual tools
Skills required	Programming and database skills	Platform knowledge and basic Deluge
Hosting & infrastructure	Requires server and database setup	Platform-managed infrastructure
Mobile application	Requires responsive or separate app development	Supports access on compatible devices
Security & access control	Requires custom implementation	Built-in security and configurable permissions
Maintenance & change	Often requires code modifications	Easy form and workflow updates
Customisation limits	High flexibility	Limited by platform capabilities
Cost	Development and infrastructure costs	Depends on subscription and usage

2.4 Literature Survey

Low-code development platforms have received increasing attention because they simplify application development through visual modelling, reusable components, and workflow automation. The following literature and technical

documentation provide a foundation for selecting Zoho Creator and designing SATGUARD.

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	Prinz, Rentrop and Huber (2021)	Low-Code Development Platforms – A Literature Review	Reviews research on low-code development platforms and identifies technical and organisational aspects of their adoption.	Supports the selection of a low-code approach for developing SATGUARD efficiently.
2	Gartner (2024)	Critical Capabilities for Enterprise Low-Code Application Platforms	Discusses the evaluation of low-code platforms across application development, connectivity, security, testing, and governance capabilities.	Provides criteria for considering platform capabilities, security, and application maintenance.
3	Zoho Corporation (n.d.)	Understanding Workflows – Zoho Creator Help	Describes form workflows, scheduled workflows, approvals, blueprints, batch workflows, and report workflows.	Supports the design of automated alert generation and maintenance tracking in SATGUARD.
4	Zoho Corporation (n.d.)	Configure Deluge Script on Success Action	Explains how Deluge scripts can execute following successful form submission and how workflow actions are configured.	Supports the implementation of telemetry checks and automatic alert creation after form submission.
5	Zoho Corporation (n.d.)	Configuring Deluge Script Actions for Report Workflows	Explains how custom Deluge scripts can be executed through report workflow actions for application-specific logic.	Provides additional guidance for implementing customised operations within the application.

2.5 Emerging Trends in Low-Code Development

Low-code development is evolving beyond basic form creation towards integrated business applications, automated workflows, and data-driven decision support. Several trends are relevant to the design and future development of SATGUARD.

- **AI-assisted application development:** AI-based tools can assist developers in generating application components, suggesting workflows, and simplifying repetitive development tasks. In future versions, AI could support the analysis of telemetry trends, subject to appropriate data and validation.
- **Workflow automation:** Modern platforms support event-triggered actions and scheduled processes that reduce repetitive manual work. SATGUARD applies this concept by generating alerts from configured telemetry conditions and recording maintenance completion dates.
- **Centralised dashboards and reporting:** Visual dashboards make operational information easier to review by presenting key metrics and records in one place. SATGUARD uses a dashboard to summarise satellite status, open alerts, and maintenance tasks.
- **Integration with external services:** Low-code platforms increasingly support connections to external applications and APIs. A future version of SATGUARD could integrate verified satellite data sources to reduce manual telemetry entry.
- **Security and governance:** Access permissions, controlled deployment, validation, and appropriate data management are important when low-code applications are used for operational processes.

These trends demonstrate the potential for expanding SATGUARD from a record-management prototype into a more integrated monitoring application. However, AI-driven anomaly detection and live external satellite-data integration are future enhancements unless separately implemented and tested.

2.6 Proposed System

SATGUARD is proposed as a centralised, low-code satellite monitoring and alert management application developed using Zoho Creator. It addresses the

limitations of manual and disconnected record-keeping by combining satellite registration, telemetry logging, alert management, maintenance tracking, and dashboard reporting in one application.

The proposed system consists of four main forms:

1. **Satellite Registry:** Maintains satellite names, mission types, launch dates, orbit types, operational statuses, and mission descriptions.
2. **Telemetry Log:** Records satellite telemetry information, including battery voltage, temperature, power level, communication status, and reading timestamps.
3. **Alert Log:** Stores automatically generated and manually recorded alerts with their types, severity levels, timestamps, descriptions, and resolution statuses.
4. **Maintenance Log:** Records maintenance tasks, assigned personnel, task status, actions taken, and resolution dates.

Deluge scripting and workflows automate important operations. Configured telemetry conditions generate alerts for communication failures, low battery voltage, high temperature, and low power levels. Duplicate-alert prevention reduces repeated open alerts for the same satellite and alert type. When a maintenance task is marked as completed, the system records its completion date. A notification workflow sends an email when a critical alert is created.

The dashboard and reports provide a consolidated view of satellite records, operational status, open alerts, and maintenance activities. Field validation helps prevent invalid entries, while the reporting features support easier retrieval and review of information.

The proposed system is designed to improve data organisation, reduce repetitive manual operations, and provide a more systematic process for managing alerts and maintenance activities. The current implementation uses entered telemetry values and predefined threshold rules; it does not directly communicate with satellites or independently verify live orbital conditions.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

SATGUARD uses role-based access control to organise application access according to user responsibilities. Custom permission sets have been created for Monitoring Staff and Maintenance Staff. The Administrator manages the application and oversees its configuration, users, workflows, and records.

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator	Manages the application and its configuration	Manage satellite records, configure workflows, manage users and permissions, and access reports and alerts.
Monitoring Staff	Responsible for recording and monitoring satellite telemetry.	View satellite details, enter telemetry readings, review alerts, and monitor satellite status.
Maintenance Staff	Responsible for tracking maintenance activities.	View relevant satellite and alert information, create and update maintenance records, record corrective actions, and complete maintenance tasks.

3.2 Functional Requirements

Functional requirements define the operations that SATGUARD must perform. Each requirement is assigned a unique identifier for reference during system testing.

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall allow the Administrator to register and maintain satellite details.	Administrator	High
FR-02	The system shall allow Monitoring Staff to enter telemetry readings, including battery voltage, temperature, power level, and communication status.	Monitoring Staff	High

FR ID	Requirement	Role(s)	Priority
FR-03	The system shall generate alerts when configured telemetry thresholds are violated or communication fails.	System	High
FR-04	The system shall prevent duplicate open alerts for the same satellite and alert type.	System	High
FR-05	The system shall maintain alert severity, timestamp, description, and resolution status.	Administrator	High
FR-06	The system shall allow Maintenance Staff to manage maintenance records and automatically record the completion date when a task is marked as completed.	Maintenance Staff	High
FR-07	The system shall send an email notification when a critical alert is created.	System	High
FR-08	The system shall provide reports and a dashboard to display satellite status, alerts, and maintenance information to authorised users.	Administrator	High
FR-09	The system shall validate telemetry inputs and reject power-level values outside the permitted range of 0 to 100.	System	High
FR-10	The system shall restrict access to forms, reports, and application components according to configured user permissions.	Administrator / System	High

3.3 Non-Functional Requirements

Non-functional requirements describe the quality attributes and operational constraints of SATGUARD.

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	The application should display records and reports within an acceptable response time under normal usage.
NFR-02	Security	Application access should be controlled using configured roles and permission sets.
NFR-03	Usability	Forms, dashboards and reports should be clear and easy to navigate.
NFR-04	Availability	The application should be accessible when the Zoho Creator service and network connection are available.

NFR ID	Category	Requirement
NFR-05	Maintainability	Forms, validation rules and workflows should be easy to maintain and modify.

3.4 Hardware and Software Requirements

SATGUARD is developed as a cloud-based low-code application using Zoho Creator. Therefore, the application does not require a dedicated local server or a separately installed database.

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator Education Edition
Scripting language	Deluge (Data Enriched Language for the Universal Grid Environment)
Browser	Google Chrome / Microsoft Edge / Mozilla Firefox (latest version)
API testing tool	Postman
Other Zoho services	Zoho CRM
Mobile	Zoho Creator mobile app (Android / iOS)
Client hardware	Intel Core i3/i5 or above, 8 GB RAM, internet connection

3.5 Use Case Model

The use case model represents the interaction between SATGUARD users and the application. The Administrator manages the application, Monitoring Staff records and reviews telemetry information, and Maintenance Staff manages maintenance activities. Automated workflows perform configured alert-generation, validation, duplicate-prevention, completion-date, and notification operations.

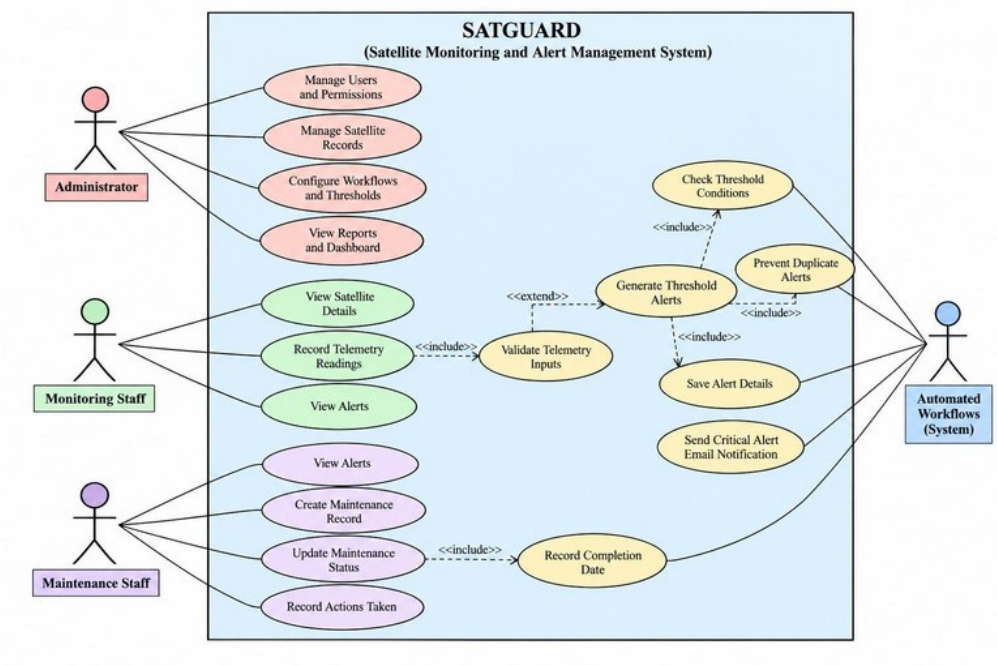


Figure 3.1: Use Case Diagram of the Application

Table 3.5: Use Case Description – Register Satellite

Use Case ID	UC-01
Use Case Name	Register Satellite
Actor(s)	Administrator
Pre-condition	The Administrator has access to the Satellite Registry form.
Main Flow	1. Open the Satellite Registry form. 2. Enter satellite details. 3. Submit the form. 4. The system saves the record.
Alternate Flow	If required information is missing or invalid, the record must be corrected before submission.
Post-condition	The satellite record is stored and available for authorised users to view.
Related FR	FR-01, FR-10

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

SATGUARD is a cloud-based satellite monitoring and alert management application developed using Zoho Creator, a low-code development platform. The architecture consists of the user interface layer, application logic layer, data storage layer, and access control layer.

The user interface layer contains the Satellite Registry, Telemetry Log, Alert Log, and Maintenance Log forms, along with reports and the dashboard. These components allow authorised users to enter information, monitor satellite conditions, review alerts, and manage maintenance activities.

The application logic layer uses Deluge scripting and Zoho Creator workflows to automate operations. These workflows validate telemetry values, detect configured threshold violations, generate alerts for communication failures, prevent duplicate open alerts, and record maintenance completion dates where configured.

The data storage layer uses Zoho Creator's built-in data storage to maintain satellite details, telemetry readings, alert records, and maintenance information. Lookup fields establish relationships between related records.

The access control layer uses configured roles and permission sets to restrict access to application components. Monitoring Staff and Maintenance Staff have separate permission sets, while the Administrator manages application configuration and oversees system operations.

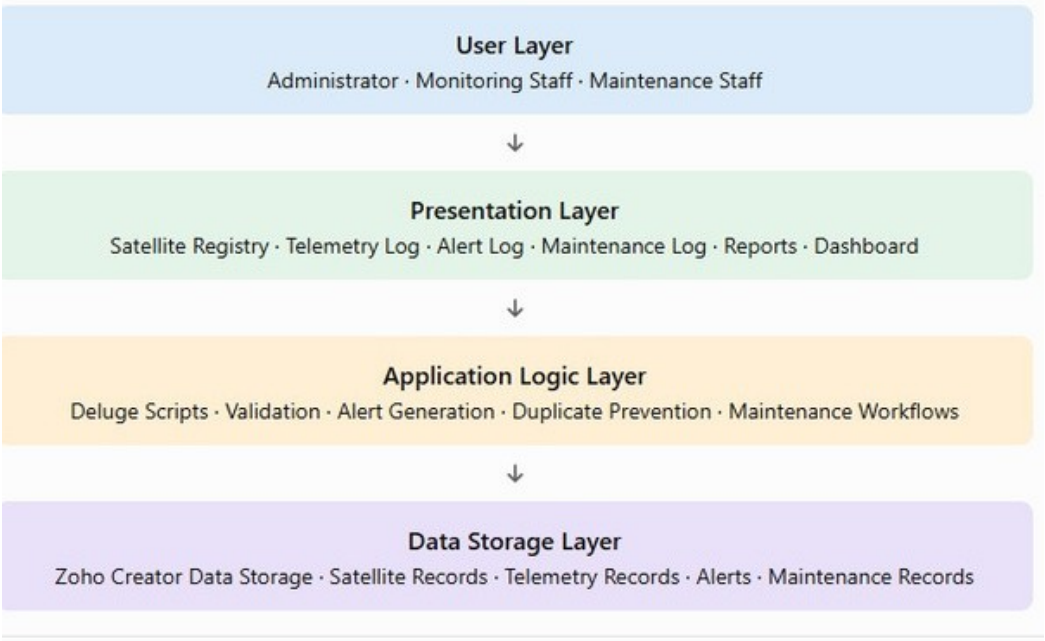


Figure 4.1: System Architecture of the Application

4.2 Data Model

The data model defines the forms used to store information and the relationships between them. SATGUARD contains four primary entities: Satellite Registry, Telemetry Log, Alert Log, and Maintenance Log.

The Satellite Registry stores the basic details of each satellite. The Telemetry Log stores readings associated with a selected satellite. The Alert Log records conditions that require attention, while the Maintenance Log tracks actions taken to investigate or resolve issues.

Lookup fields connect related records. A satellite can have multiple telemetry records and alerts, and an alert can be associated with maintenance records.

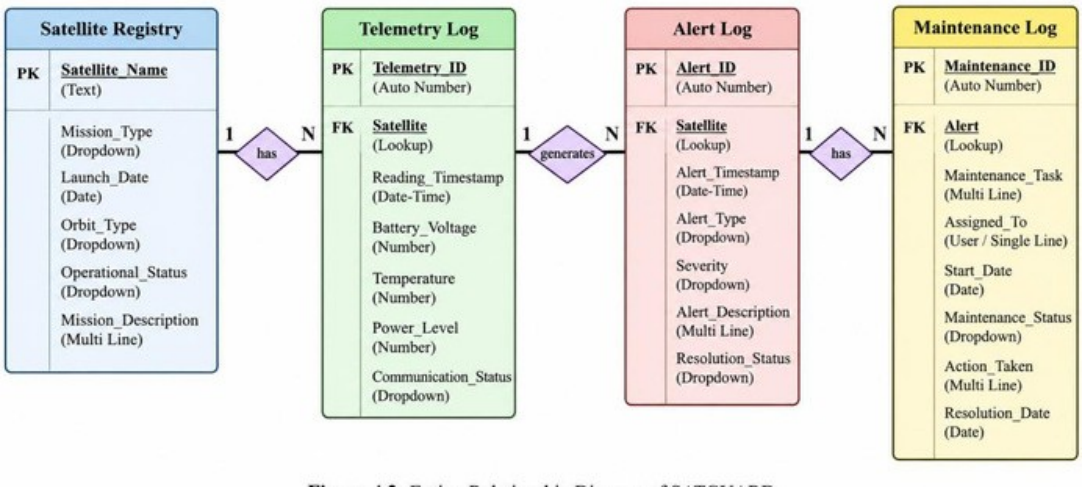


Figure 4.2: Entity–Relationship Diagram

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Satellite Registry	Satellite name, mission type, orbit type, status	Telemetry Log, Alert Log	One-to-many	Lookup field in the related forms
Telemetry Log	Satellite lookup, timestamp, voltage, temperature, power	Satellite Registry	Many-to-one	Lookup field (Satellite)
Alert Log	Satellite lookup, alert type, severity, status	Satellite Registry, Maintenance Log	Many-to-one with Satellite Registry; one-to-many with Maintenance Log	Lookup field (Satellite)
Maintenance Log	Alert lookup, task, assigned person, status, resolution date	Alert Log	Many-to-one	Lookup field (Alert_Reference)

4.3 Form Design

SATGUARD uses structured forms to capture, validate, and maintain application data. Each form contains fields appropriate to its purpose. Mandatory fields and field types help maintain consistency, while lookup fields connect records across forms.

Table 4.2: Field Specification – Satellite Registry

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Satellite_Name	Satellite_Name	Single Line	Yes	No duplicate values	Unique name of the satellite
Mission_Type	Mission_Type	Dropdown	Yes	–	Mission category
Launch_Date	Launch_Date	Date	Yes	–	Satellite launch date
Orbit_Type	Orbit_Type	Dropdown	Yes	–	Orbit classification
Operational_ Status	Operational_ Status	Dropdown	Yes	–	Current operational status
Mission_ Description	Mission_ Description	Multi Line	No	–	Description of the mission

4.4 Process and Workflow Design

SATGUARD uses Deluge scripts and Zoho Creator workflows to automate data validation, alert generation, duplicate prevention, and maintenance tracking. Workflows are triggered by relevant form submissions or updates.

4.4.1 Telemetry Validation Process

When a user submits a telemetry record, the system checks the power-level value. If the value is below 0 or above 100, the submission is rejected and an alert message is displayed. Valid values can proceed through the submission process.

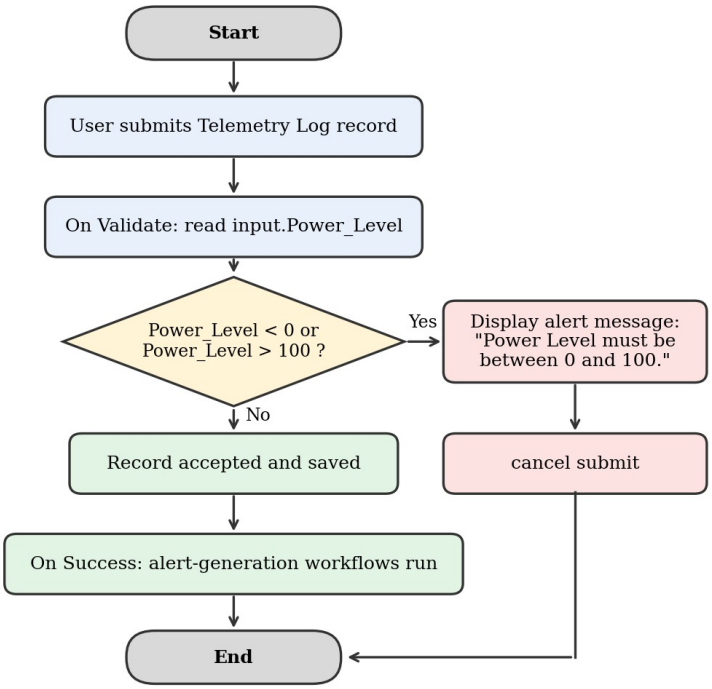


Figure 4.3: Workflow / Process Flow for Telemetry Validation

4.4.2 Automated Alert Generation

The alert-generation workflow evaluates telemetry readings against configured thresholds. It also checks communication status. When a condition is met, the system creates an alert with the appropriate type, severity, timestamp, description, and resolution status. Duplicate-prevention logic checks for an existing open alert for the same satellite and alert type before creating another.

The current prototype uses configurable demonstration thresholds. These values are application settings for testing and should not be interpreted as universal spacecraft operating limits.

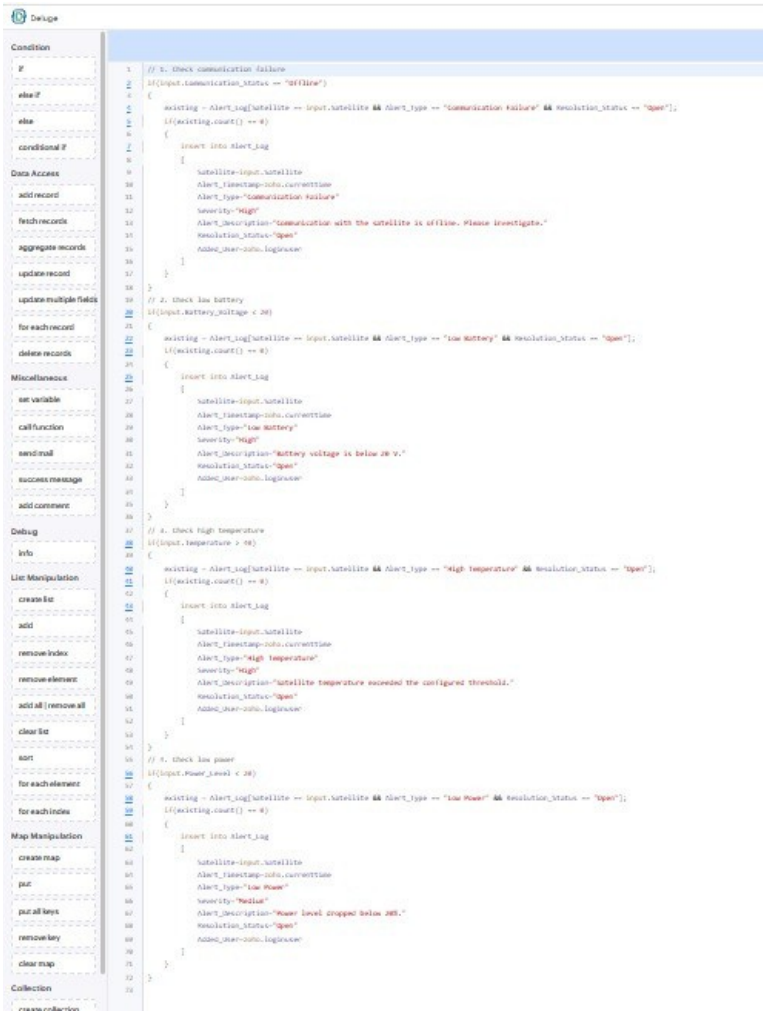


Figure 4.4: Deluge Script for Automated Alert Generation

4.4.3 Maintenance Completion Workflow

When a maintenance record is updated to Completed, the workflow records the completion date if it has not already been populated. This provides a reference for tracking maintenance activities and their completion.



Figure 4.5: Deluge Script for Maintenance Completion Date

4.5 User Interface and Navigation Design

SATGUARD provides a centralised interface for managing satellite information, telemetry, alerts, and maintenance activities. The interface is organised around forms for data entry, reports for record viewing, and a dashboard for summarising important information.

The main navigation components are:

- **Satellite Registry:** Adds and displays satellite details.
- **Telemetry Log:** Records battery voltage, temperature, power level, and communication status.
- **Alert Log:** Displays generated alerts and their severity and resolution status.
- **Maintenance Log:** Tracks maintenance tasks, assigned personnel, corrective actions, and completion dates.
- **Reports:** Provides access to stored records in suitable report formats.
- **Dashboard:** Summarises satellite counts, operational status, open alerts, maintenance tasks, and relevant charts.

Access to these components is controlled by the configured permission sets. Monitoring Staff primarily work with telemetry and alerts, while Maintenance Staff work with maintenance records. Administrative functions are reserved for authorised administrators.

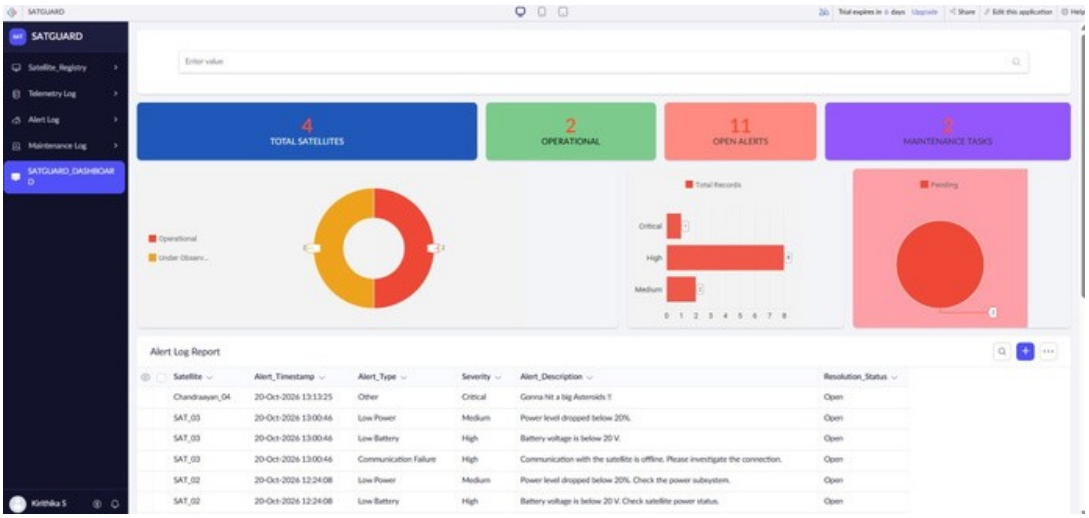


Figure 4.6: Navigation Structure and Wireframe of Dashboard

4.6 Access Control Design

SATGUARD implements a role-based access approach through Zoho Creator's user management, roles, and permission sets. The purpose is to ensure that users can access the forms and reports required for their responsibilities without unnecessarily exposing editing functions.

The two custom permission sets created are Monitoring Staff and Maintenance Staff. The Administrator oversees the application and its configuration.

Table 4.3: Role–Permission Matrix

Component	Administrator	Monitoring Staff	Maintenance Staff
Satellite Registry	Manage	View	View
Telemetry Log	Manage	View, Add	No access
Alert Log	Manage	View	View
Maintenance Log	Manage	No access	View, Add, Edit
Dashboard and Reports	Manage / View	View, where permitted	View, where permitted

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

SATGUARD was developed using Zoho Creator, a cloud-based low-code application development platform. The platform provides visual tools for creating forms, configuring reports, building dashboards, defining user permissions, and implementing automated workflows using Deluge scripting.

The application was organised into four main data-entry forms: Satellite Registry, Telemetry Log, Alert Log, and Maintenance Log. Reports and dashboard components were configured to support monitoring and record management. Workflows were added to validate telemetry data, generate alerts when configured conditions are detected, prevent duplicate open alerts, and update maintenance completion information.

The application was configured through Zoho Creator's design and workflow interfaces. Its cloud-based architecture reduces the need to configure a separate application server or database.

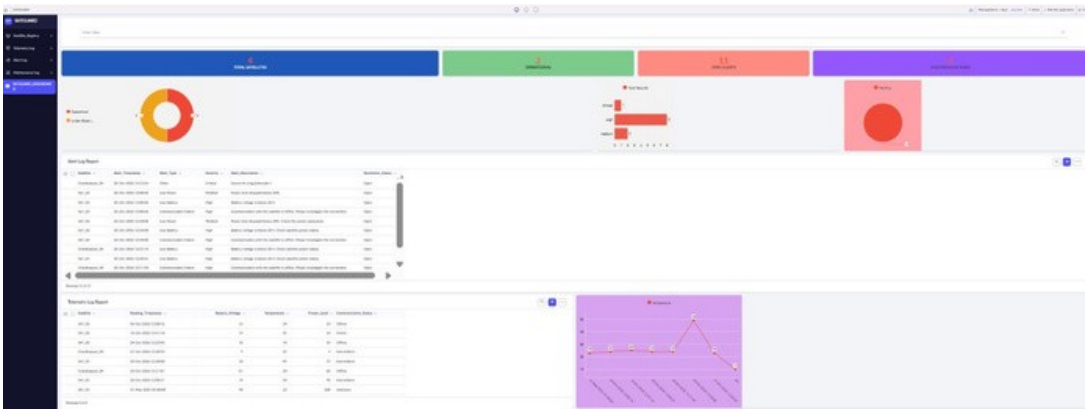


Figure 5.1: Application Home and Navigation Sections

5.2 Forms and Data Capture

SATGUARD uses structured forms to collect and maintain satellite-related information. The forms use suitable field types, dropdowns, numeric fields, date fields, and lookup fields to organise the data. Mandatory-field settings and validation rules help improve data quality.

5.2.1 Satellite Registry

The Satellite Registry stores information about each satellite, including its name, mission type, launch date, orbit type, operational status, and mission description. This form provides the main reference for linking telemetry and alert records to the corresponding satellite.

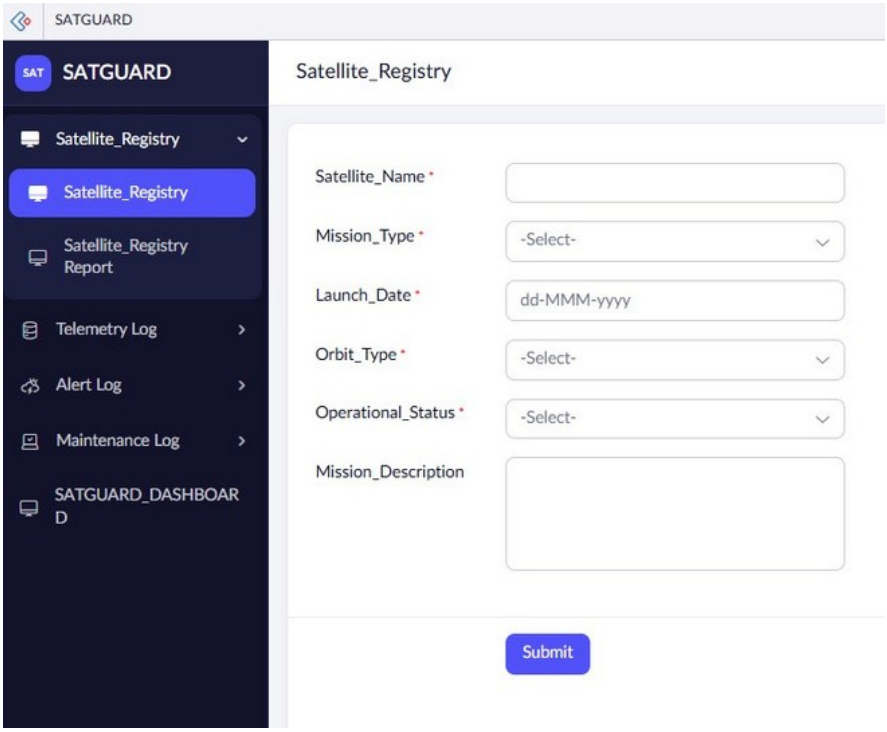


Figure 5.2: Satellite Registry Form in Live Mode

5.2.2 Telemetry Log

The Telemetry Log records telemetry readings associated with a registered satellite. Its fields include the satellite lookup, reading timestamp, battery voltage, temperature, power level, and communication status. The form acts as the input source for the automated monitoring workflows.

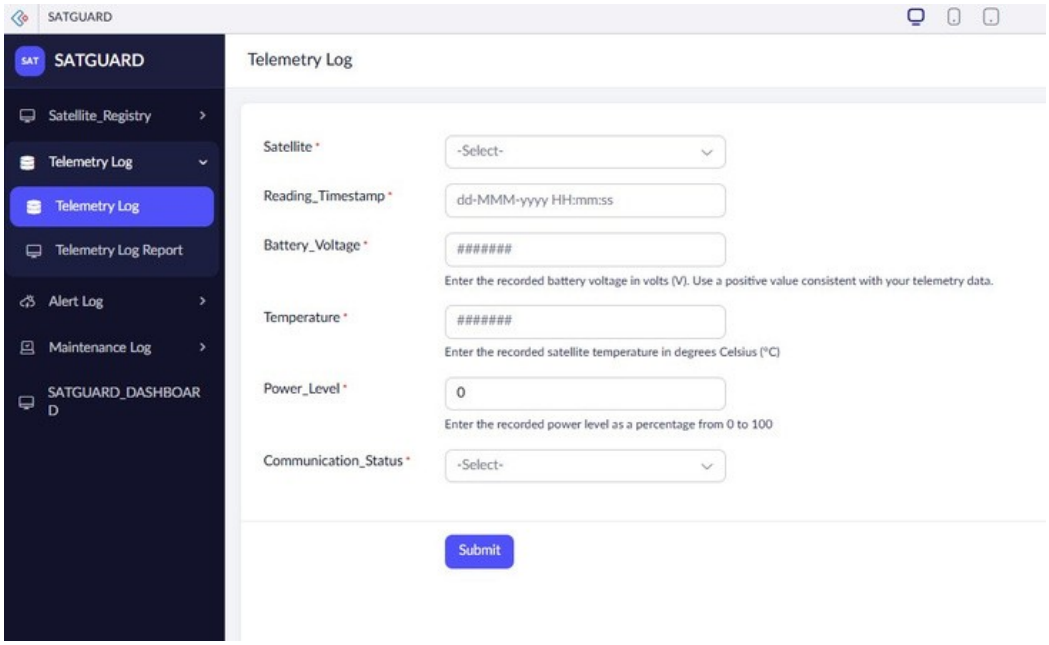


Figure 5.3: Telemetry Log Form in Live Mode

5.2.3 Alert Log

The Alert Log stores alerts generated from configured telemetry conditions. Each alert contains the associated satellite, timestamp, alert type, severity, description, and resolution status. These records allow users with the relevant permissions to review and track detected conditions.

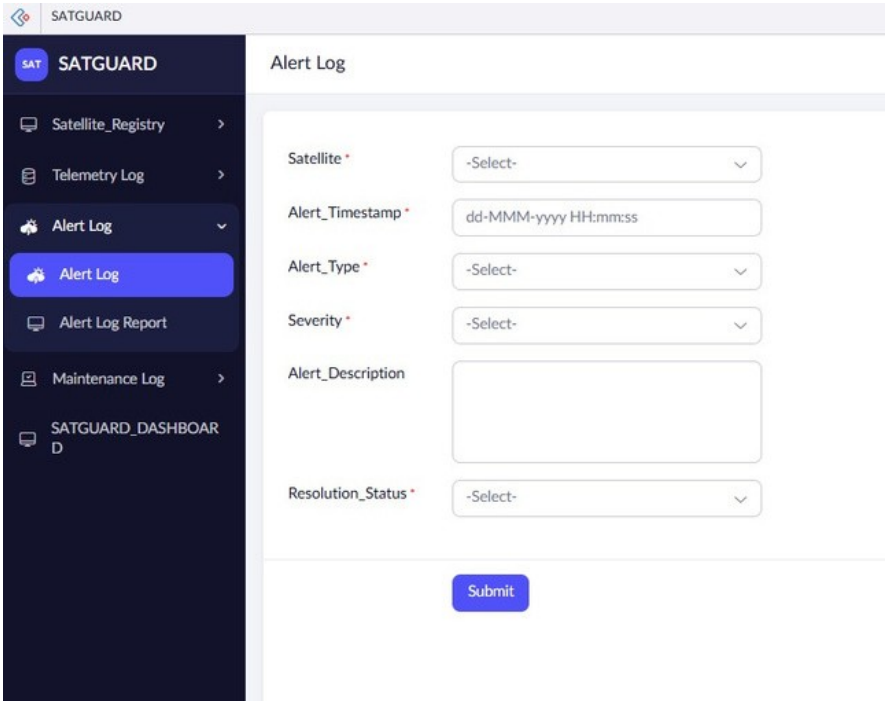


Figure 5.4: Alert Log Form in Live Mode

5.2.4 Maintenance Log

The Maintenance Log stores maintenance tasks associated with alerts. It records the maintenance task, assigned person, start date, status, actions taken, and resolution date. A configured workflow records the completion date when a task is marked as completed, provided the date is empty.

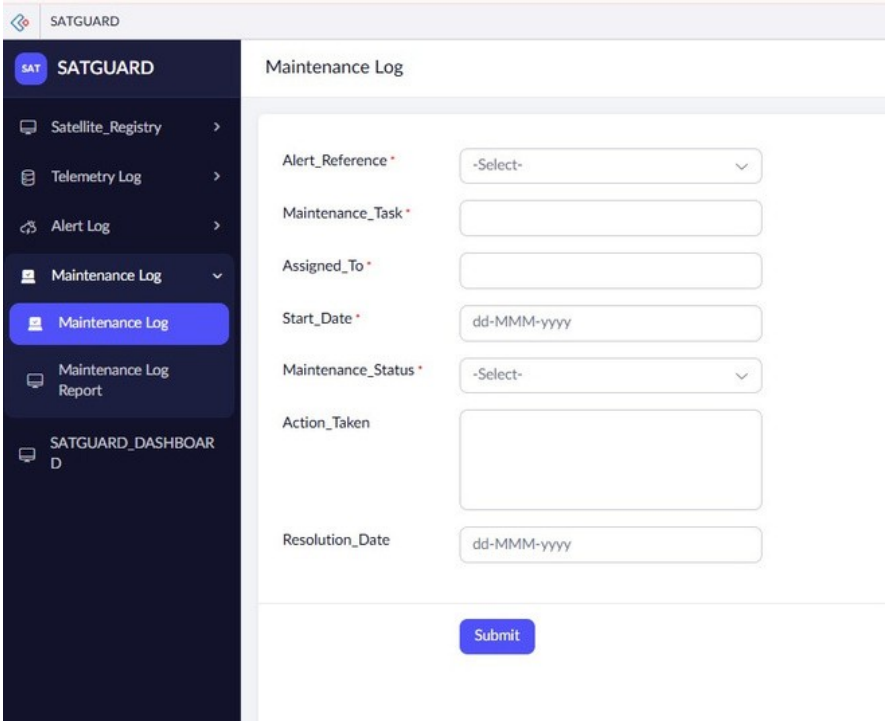


Figure 5.5: Maintenance Log Form in Live Mode

5.3 Reports

Zoho Creator reports provide structured views of the records stored in SATGUARD. They help authorised users review satellite details, telemetry readings, alerts, and maintenance activities without opening each record individually.

The application includes reports associated with the forms and a dashboard for summarising key information. Report availability and actions depend on the user's configured permissions.

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Purpose
Satellite Registry Report	List	Satellite Registry	View registered satellite details

Report Name	Type	Source Form	Purpose
Telemetry Log Report	List	Telemetry Log	Review telemetry readings
Alert Log Report	List	Alert Log	Review generated alerts and their statuses
Maintenance Log Report	List	Maintenance Log	Track maintenance activities and completion status
SATGUARD Dashboard	Dashboard	Multiple forms / reports	Summarise satellite status, open alerts, and maintenance tasks

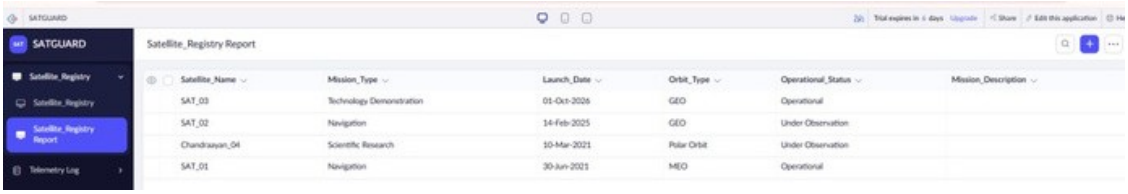


Figure 5.6: Satellite Registry Report

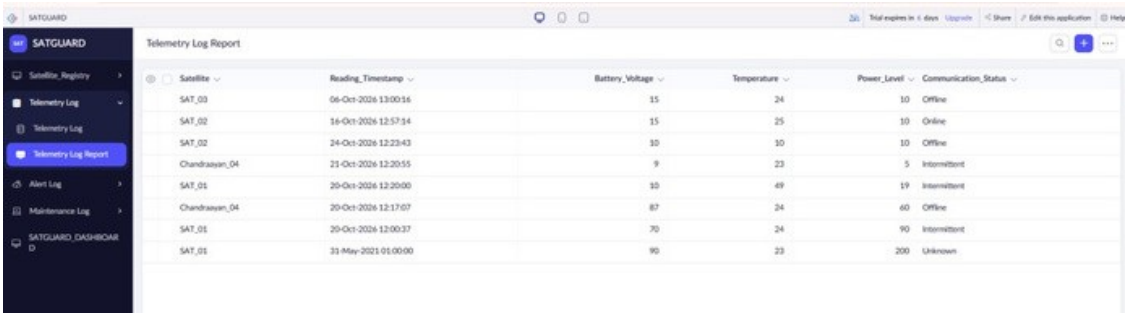


Figure 5.7: Telemetry Log Report

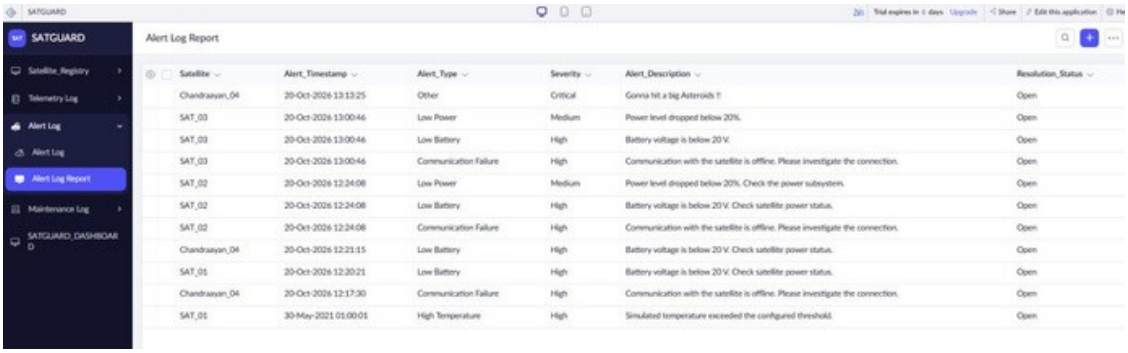


Figure 5.8: Alert Log Report

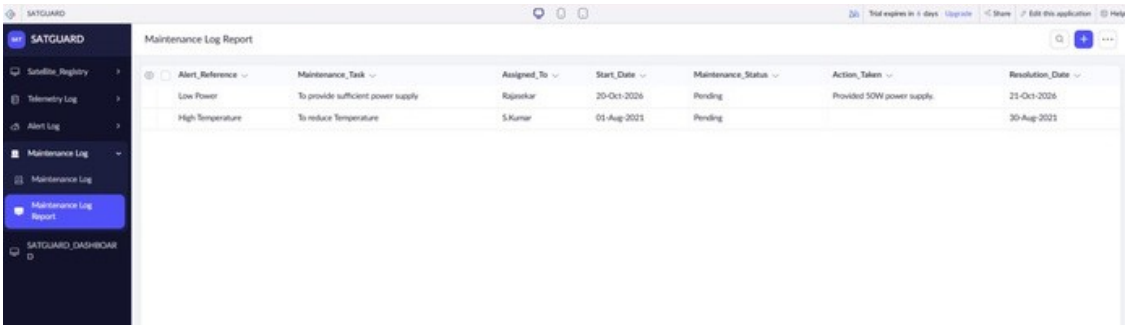


Figure 5.9: Maintenance Log Report

5.4 Pages and Dashboards

The SATGUARD dashboard provides a consolidated view of important application information. It is designed to help users understand the current state of the records without manually checking every form.

The dashboard contains summary information such as:

- Total number of registered satellites.
- Number of operational satellites.
- Number of open alerts.
- Number of maintenance tasks.
- Charts or reports showing relevant satellite and alert information, where configured.

The dashboard supports quick review and improves visibility into the records stored in the application. Its displayed information depends on the underlying records and the components included in the dashboard.

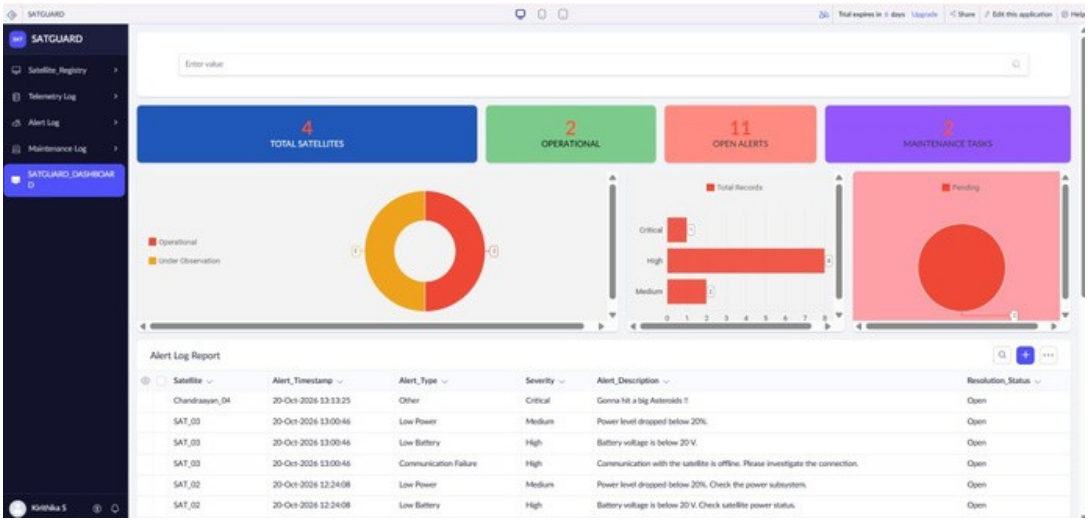


Figure 5.10: SATGUARD Dashboard

5.5 Deluge Scripting

Deluge is used in SATGUARD to implement validation and automate actions that would otherwise require manual checking. The scripts are associated with form events or workflows and execute when the configured conditions are met.

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Telemetry Log → On Validate	Validate power level	if, comparison operators, alert, cancel submit
DS-02	Telemetry Log → On Success	Generate communication failure alert	if, record criteria, insert into
DS-03	Telemetry Log → On Success	Generate low battery, high temperature and low power alerts	Conditional statements, comparison operators, insert into, field assignment
DS-04	Telemetry Log → On Success	Prevent duplicate open alerts	Fetch records with criteria, count(), if
DS-05	Maintenance Log → Configured workflow event	Record maintenance completion date	if, null check, date functions
DS-06	Alert Log → Configured workflow event	Create a Case in Zoho CRM from an alert	Map collection, string concatenation, zoho.crm.createRecord, info

Code 5.1: Power-level Validation

A validation script checks whether the power level is within the permitted range of 0 to 100. If the value is outside this range, the submission is cancelled and the user receives a validation message.

```
if(input.Power_Level < 0 || input.Power_Level > 100)
{
    alert "Power Level must be between 0 and 100.";
    cancel submit;
}
```

Code 5.2: Communication Failure Alert

The communication monitoring workflow checks the communication status recorded in the Telemetry Log. When the status is Offline, it creates a Communication Failure alert with the configured severity and an open resolution status.

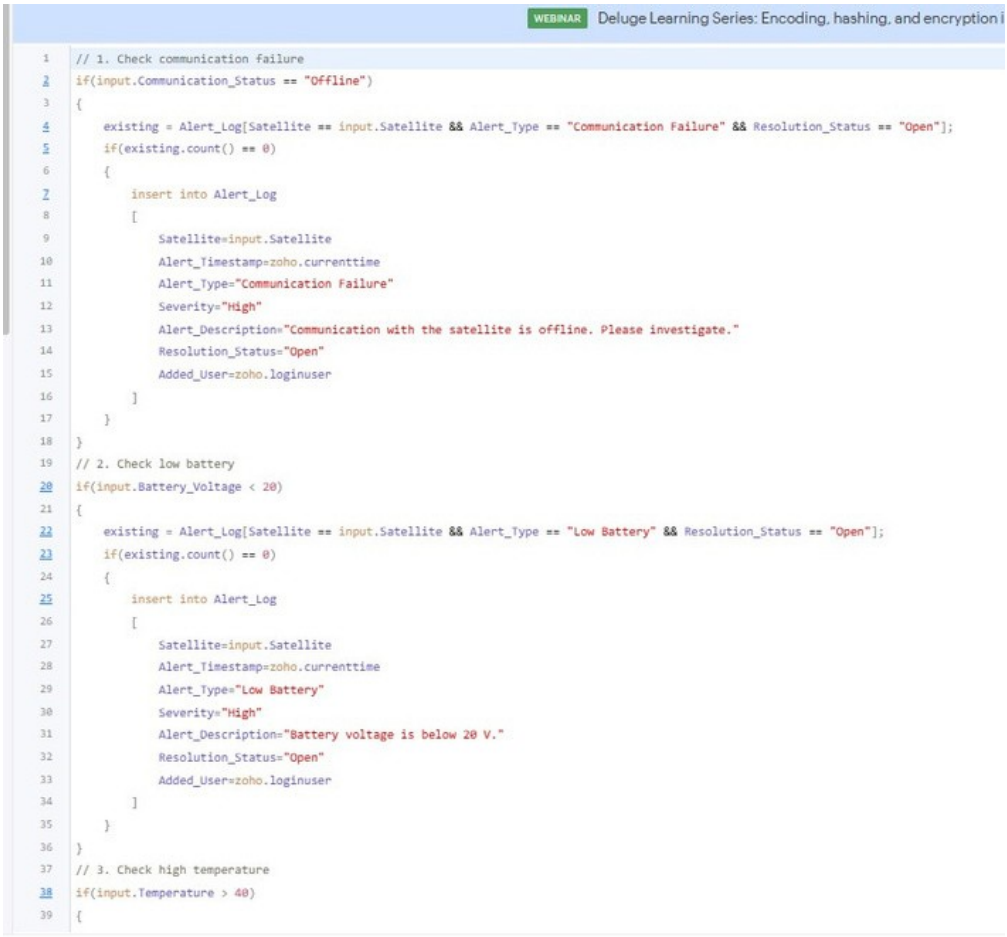
WEBINAR Deluge Learning Series: Encoding, hashing, and encryption

```
if(input.Communication_Status == "Offline")
{
    insert into Alert_Log
    [
        Satellite=input.Satellite
        Alert_Timestamp=zoho.currenttime
        Alert_Type="Communication Failure"
        Severity="High"
        Alert_Description="Communication with the satellite is offline. Please investigate the connection."
        Resolution_Status="Open"
        Added_User=zoho.loginuser
    ]
}
```

Code 5.3: Telemetry Threshold Alerts

The threshold workflows evaluate the configured telemetry values. In the current prototype, conditions such as low battery voltage, high temperature, and low power level can trigger corresponding alert records. Each alert stores its type, severity, timestamp, description, and resolution status.

The threshold values are demonstration settings for this project and should not be interpreted as universal spacecraft safety limits.



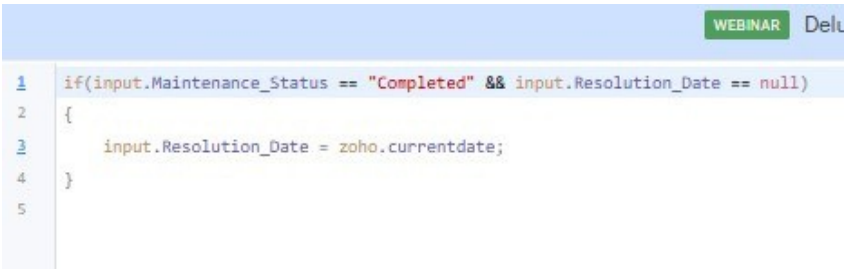

```
// 1. Check communication failure
if(input.Communication_Status == "Offline")
{
    existing = Alert_Log[Satellite == input.Satellite && Alert_Type == "Communication Failure" && Resolution_Status == "Open"];
    if(existing.count() == 0)
    {
        insert into Alert_Log
        [
            Satellite=input.Satellite
            Alert_Timestamp=zoho.currenttime
            Alert_Type="Communication Failure"
            Severity="High"
            Alert_Description="Communication with the satellite is offline. Please investigate."
            Resolution_Status="Open"
            Added_User=zoho.loginuser
        ]
    }
}
// 2. Check low battery
if(input.Battery_Voltage < 20)
{
    existing = Alert_Log[Satellite == input.Satellite && Alert_Type == "Low Battery" && Resolution_Status == "Open"];
    if(existing.count() == 0)
    {
        insert into Alert_Log
        [
            Satellite=input.Satellite
            Alert_Timestamp=zoho.currenttime
            Alert_Type="Low Battery"
            Severity="High"
            Alert_Description="Battery voltage is below 20 V."
            Resolution_Status="Open"
            Added_User=zoho.loginuser
        ]
    }
}
// 3. Check high temperature
if(input.Temperature > 40)
{
```

Code 5.4: Duplicate-Alert Prevention

Before creating an alert, the duplicate-prevention logic checks whether an open alert already exists for the same satellite and alert type, as shown in Code 5.3. If an equivalent open alert is found, another alert is not created. This helps keep the Alert Log organised when repeated telemetry submissions contain the same abnormal condition.

Code 5.5: Maintenance Completion Date

The maintenance workflow records the completion date when a maintenance task is marked as Completed, if the completion date has not already been entered. This reduces manual data entry and helps track the completion of maintenance activities.



5.6 Workflows and Automation

SATGUARD uses Zoho Creator workflows to execute predefined actions in response to form submissions or updates. These workflows reduce repetitive manual work and ensure that configured monitoring rules are applied consistently.

Table 5.3: Workflows Implemented

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Power-Level Validation	Form workflow	Telemetry submission	Power level is below 0 or above 100	Display validation message and cancel submission	DS-01
Communication Failure Alert	Form workflow	Telemetry submission	Communication status is Offline	Create a communication failure alert	DS-02
Telemetry Threshold Alerts	Form workflow	Telemetry submission	A configured threshold is violated	Create the relevant alert	DS-03
Duplicate-Alert Prevention	Form workflow / Deluge logic	Before creating an alert	Equivalent open alert exists	Avoid creating a duplicate	DS-04
Maintenance Completion Date	Form workflow	Maintenance record update	Status is Completed and completion date is empty	Populate the completion date	DS-05
Critical Alert Notification	Form workflow	Alert Log successful submission	Severity is Critical	Send email notification	–
CRM Case Creation	Form workflow	Alert Log successful submission	Critical alert created	Create a Case in Zoho CRM	DS-06

The Critical Alert Notification workflow runs on successful submission of an Alert Log record and sends an email only when the severity is Critical.

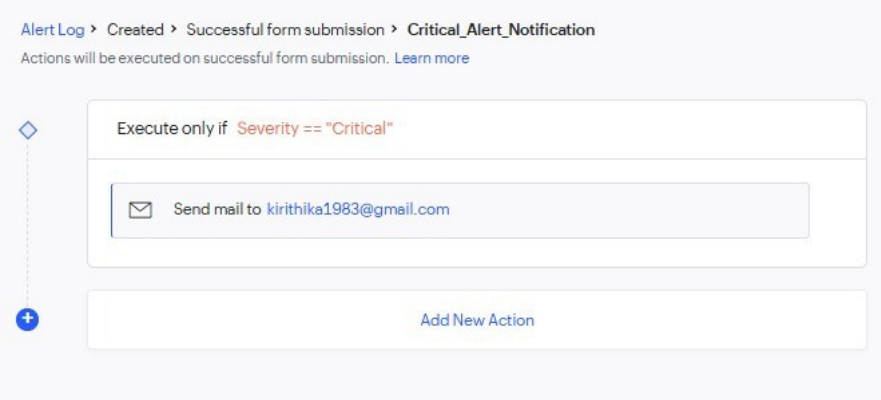


Figure 5.11: Critical Alert Notification Workflow

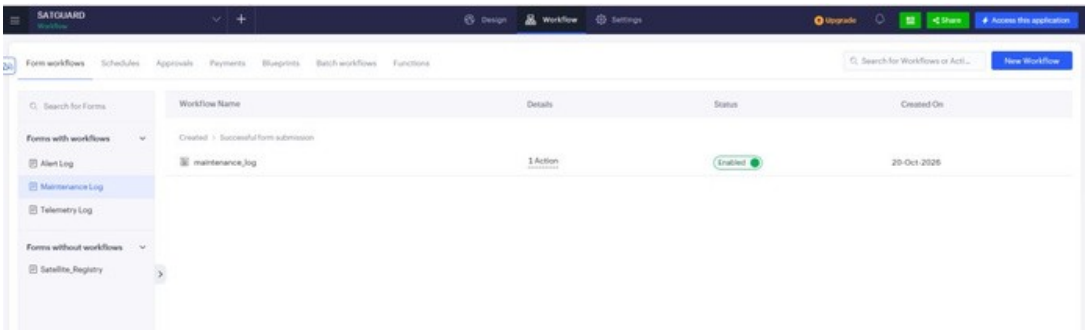


Figure 5.12: Workflow Configuration

5.7 Integrations

SATGUARD is integrated with Zoho CRM to extend its satellite alert management capabilities. Zoho Creator provides the application forms, data storage, and workflow automation, while Zoho CRM provides a platform for managing cases generated from critical satellite alerts. The integration is configured through a Zoho CRM connection in Zoho Creator.

Table 5.4: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
Zoho CRM Integration	Zoho CRM – Cases module	Creator workflow action to create a Case	Authorised Zoho CRM connection	Sends critical alert details to CRM

Code 5.6: Zoho CRM Integration Script

The script collects the alert details into a map, builds the case description, and creates a record in the Cases module through the authorised connection.

```
// SATGUARD → Zoho CRM: Create a Case from an Alert

caseDetails = Map();

caseDetails.put("Subject", "SATGUARD Alert - " + input.Alert_Type);
caseDetails.put("Case_Origin", "Web");
caseDetails.put("Status", "New");

descriptionText = "Satellite: " + input.Satellite
    + "\nAlert Type: " + input.Alert_Type
    + "\nSeverity: " + input.Severity
    + "\nAlert Timestamp: " + input.Alert_Timestamp
    + "\nResolution Status: " + input.Resolution_Status
    + "\nDescription: " + input.Alert_Description;

caseDetails.put("Description", descriptionText);

// Replace the connection link name with the exact
// link name from your Zoho Creator connection settings.
response = zoho.crm.createRecord(
    "Cases",
    caseDetails,
    Map(),
    "https://creator.zoho.in/userhome/kirithika1983/admindashboard#/microservices/connection/internal_zohocrmE"
);

info response;
```

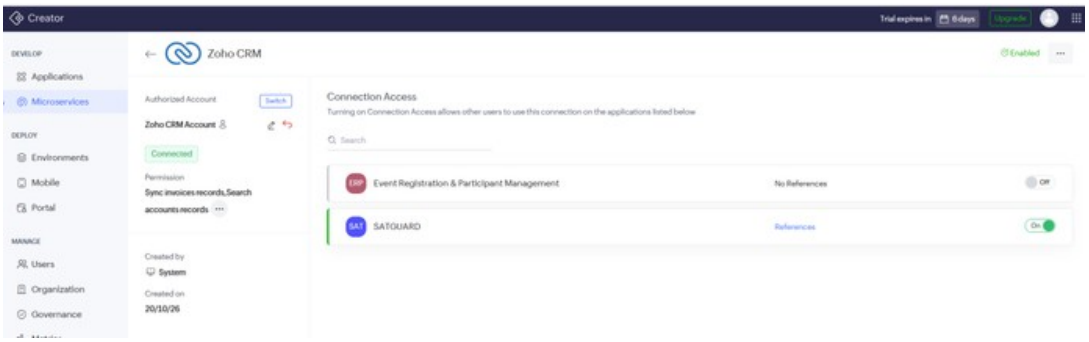


Figure 5.13: Zoho CRM Connection Setup

5.8 User Management and Role-Based Access

SATGUARD uses Zoho Creator's user management and permission configuration to restrict access to application components according to user responsibilities. Permission profiles have been created for Monitoring Staff and Maintenance Staff, while the administrator manages the application configuration and overall data.

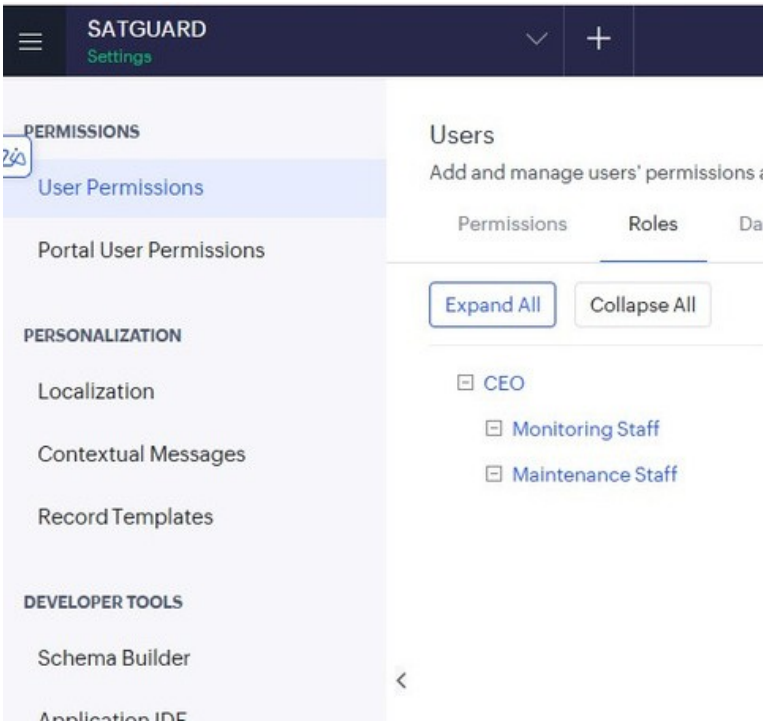


Figure 5.14: Roles

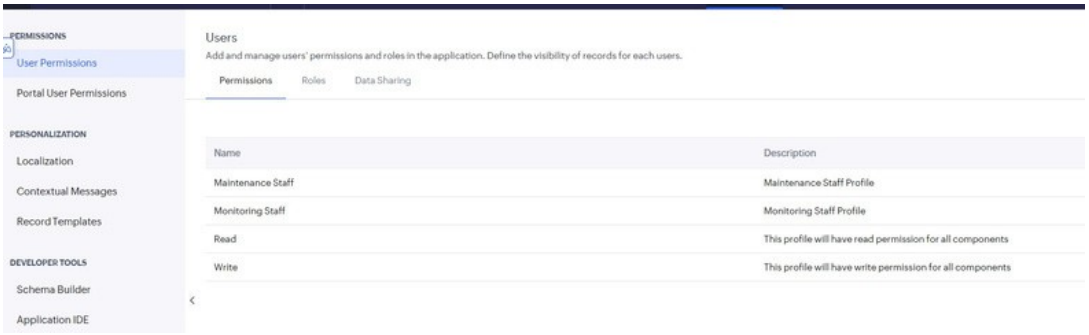


Figure 5.15: Permissions

5.9 AI and Analytics Features

SATGUARD focuses on rule-based satellite telemetry monitoring and workflow automation rather than AI-based prediction. The application records battery voltage, temperature, power level, and communication status through the Telemetry Log form. Configured Deluge workflows evaluate these values and generate alerts when specified conditions are met.

The dashboard and reports provide a consolidated view of satellite records, operational status, open alerts, and maintenance activities. These reporting features help users review system information and identify records that require attention.

The current implementation uses predefined thresholds and conditional logic; it does not claim machine-learning-based anomaly detection, AI-generated predictions, or external AI analytics. The configured thresholds are prototype settings and should not be interpreted as universal spacecraft safety limits.

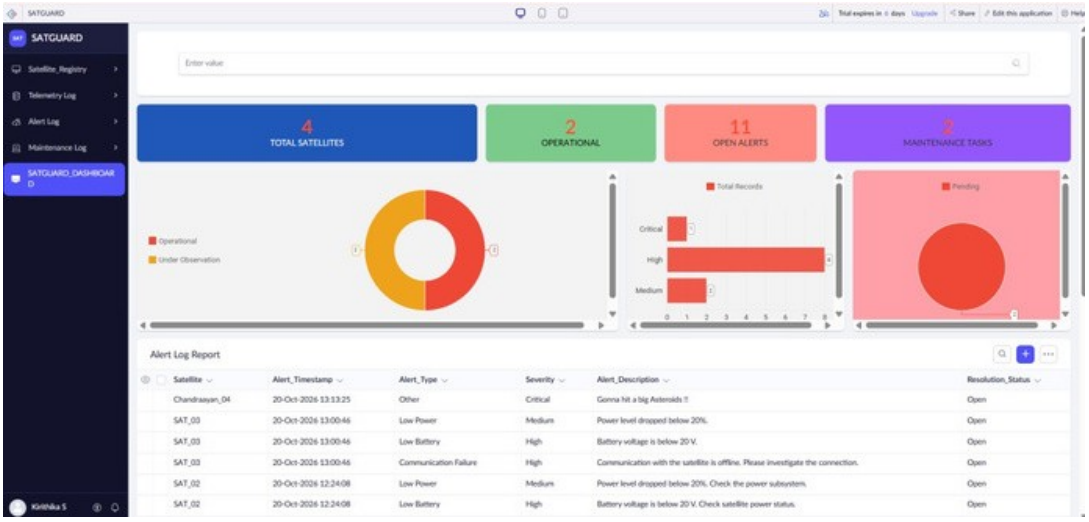


Figure 5.16: Dashboard Analytics Components

5.10 Pro-Code Extension

SATGUARD was developed primarily using Zoho Creator's low-code tools, including visual form builders, reports, workflows, and Deluge scripting. Deluge scripts were used to implement telemetry validation, threshold-based alert generation, duplicate-alert prevention, and maintenance-related automation.

The Zoho CRM integration was configured using the platform's built-in connection and workflow action. A separate custom REST API implementation or Zoho Catalyst extension was not required for the current design. This approach reduces the need to develop and maintain a separate backend for the low-code application.

5.11 Mobile and Portal Access

SATGUARD is developed on Zoho Creator, a cloud-based application platform. Users can access the application through a supported web browser, subject to their assigned permissions and the availability of the service. The form-based interface allows authorised users to enter telemetry readings, review alerts, and update maintenance records.

The application also provides reports and dashboard views for monitoring satellite status and related activities. A separate customer portal or dedicated mobile-specific interface has not been documented as part of the current implementation. Mobile access should therefore be described as browser or platform-supported access, subject to the account's enabled features and responsive behaviour.



Figure 5.17: Application on Mobile

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing was carried out to verify the functionality, reliability, data validation, workflow automation, external integration, and access control of the SATGUARD application. The testing strategy included the following levels:

- **Form Testing:** Verified mandatory fields, field types, dropdown selections, and data submission in Satellite Registry, Telemetry Log, Alert Log, and Maintenance Log.
- **Workflow Testing:** Checked whether telemetry conditions trigger the appropriate alerts and whether duplicate open alerts are prevented.
- **Integration Testing:** Checked the connection between Zoho Creator and Zoho CRM for creating Cases from critical alerts.
- **Role-Based Access Testing:** Verified that Monitoring Staff and Maintenance Staff have access only to the forms, reports, and operations allowed by their permission profiles.
- **Validation Testing:** Tested invalid Power Level values to ensure that values below 0 or above 100 are rejected.
- **User Acceptance Testing:** Evaluated whether users could perform their assigned activities through the application interface.

Testing was performed using sample satellite records and telemetry readings. The results were compared with the expected behaviour of the application, and any errors encountered were investigated and corrected where possible.

6.2 Test Cases

Table 6.1 lists the test cases designed to verify the functional and non-functional requirements of SATGUARD.

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-01	Register a satellite	Enter all mandatory satellite details and submit	Satellite record is saved	To be recorded	Pending

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-02	FR-01	Submit satellite form with missing mandatory data	Leave a required field empty and submit	Validation message is displayed	To be recorded	Pending
TC-03	FR-02	Record telemetry readings	Enter valid telemetry values and submit	Telemetry record is saved	To be recorded	Pending
TC-04	FR-03	Test communication failure alert	Set Communication Status to Offline	High-severity communication alert is created	To be recorded	Pending

6.3 Debugging Deluge Scripts

Debugging was used to identify and correct errors in the Deluge scripts and workflow configurations. The following techniques were considered:

- **Info statements:** Used to display variable values and API responses during script execution.
- **Validation checks:** Used to test boundary values and prevent invalid data submissions.
- **Workflow verification:** Checked the trigger event, execution condition, and configured actions.
- **Integration response checking:** Inspected Zoho CRM responses to identify connection, permission, mandatory-field, or invalid-value errors.
- **Incremental testing:** Tested individual workflow conditions separately before checking the complete automation.

During testing, a critical alert created in the Alert Log generated the notification email shown below.

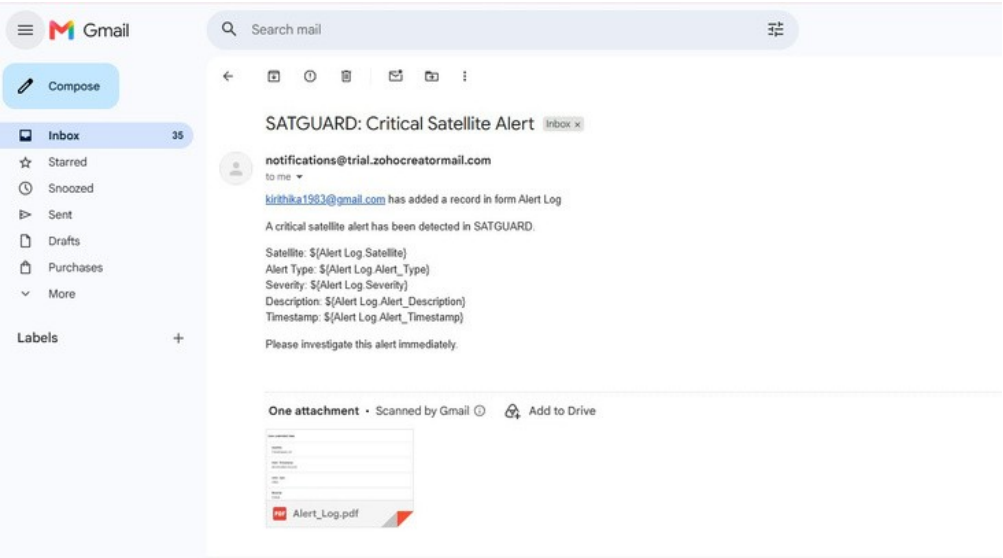


Figure 6.1: Critical Alert Email Notification

6.4 Defect Log

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-01	Invalid Power Level values were accepted during initial testing	Validation rule was not yet configured	Added Deluge validation for values outside 0–100	Closed
BUG-02	More than one alert could be generated for repeated telemetry submissions	Duplicate prevention was not included in the initial workflow	Added a check for an existing open alert before creating another	Verify after testing
BUG-03	Zoho CRM Case creation failed	A full Creator URL was entered where a connection link name was required	Replace it with the correct connection link name and validate the script	Open until verified

6.5 User Acceptance Testing

User Acceptance Testing (UAT) checks whether SATGUARD supports the activities required by its intended users. Testing should be performed by users with the appropriate assigned permission profiles.

Table 6.3: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken
User 1	Monitoring Staff	To be collected after testing telemetry entry and alert viewing	To be recorded
User 2	Maintenance Staff	To be collected after testing maintenance record updates	To be recorded

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

SATGUARD was developed using Zoho Creator, a cloud-based low-code application development platform. The application was configured using forms, reports, dashboards, Deluge scripts, workflows, and role-based access permissions.

The application was tested in the development environment to verify form submissions, telemetry validation, automated alert generation, maintenance records, reports, and dashboard functionality. Zoho Creator's environment and publishing features were used to prepare and deploy the application.

The Zoho CRM connection was configured to enable communication between SATGUARD and Zoho CRM. The integration was designed to create CRM case records from satellite alerts. The successful creation of a case must be verified before reporting the integration as fully functional.

The application was published and deployed through Zoho Creator. Before production use, the configured permissions, workflows, CRM connection, and application URL should be checked to ensure that the deployed version behaves as expected.

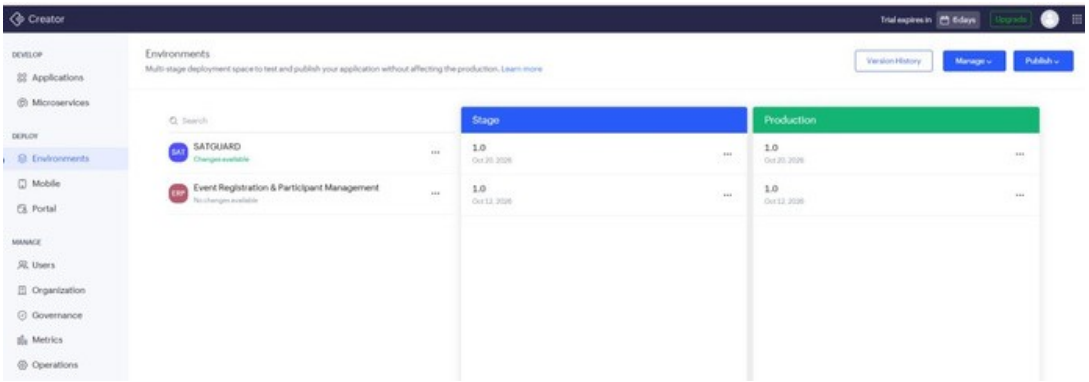


Figure 7.1: Environments

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done (✓)
1	All test cases executed in Development / Stage	✓
2	Sample/test data removed or isolated	✓

S.No.	Checklist Item	Done (✓)
3	Roles, permissions and users configured in Production	✓
4	Connections authorised in Production	✓
5	Schedules and workflows enabled	✓
6	Application published and URL verified	✓
7	Version / release notes recorded	✓

7.2 Performance Optimisation

Performance optimisation was considered during the development of SATGUARD to improve the handling of satellite records, telemetry readings, alerts, and maintenance information.

Zoho Creator reports and dashboards were used to organise application data and provide convenient access to important information. Reports were configured to display relevant records, while dashboard components summarise satellite status, open alerts, and maintenance activities.

Deluge workflows were used to automate repetitive tasks, including telemetry threshold checking and alert creation. Input validation was implemented to prevent invalid power-level values from being submitted.

The alert-generation logic was designed to check different telemetry conditions independently so that multiple alerts can be generated when more than one threshold is violated. Duplicate-prevention logic should also be tested to ensure that an existing open alert is not unnecessarily recreated.

Formal performance measurements, such as response time under concurrent users or processing time for large datasets, were not established as part of the documented implementation. Therefore, numerical performance improvements should only be reported after measurement.

Table 7.2: Performance Optimisation Results

Optimisation Applied	Component	Before	After	Improvement
Input validation for power level	Telemetry Log	Invalid values could be submitted without the configured rule	Values outside 0–100 are rejected	Improved data integrity

Optimisation Applied	Component	Before	After	Improvement
Automated telemetry threshold checking	Telemetry workflows	Manual checking required	Matching conditions can generate alerts automatically	Reduced manual effort
Separate condition checks for telemetry alerts	Alert-generation workflow	An if/else-if structure could create only one matching alert	Independent conditions allow multiple matching alerts	Improved alert coverage

7.3 Security Features

Security in SATGUARD is managed through the access-control features available in Zoho Creator and the authentication mechanisms used for external integrations.

Application roles and permissions were configured to control access to the available forms, reports, dashboards, and application functions. These permissions help separate administrative responsibilities from routine monitoring and maintenance activities.

Mandatory fields, field types, and validation rules help maintain the accuracy of application records. In particular, the power-level validation prevents users from submitting values below 0 or above 100.

The Zoho CRM integration uses an authorised connection to access the external service. This avoids the need to embed CRM authentication credentials directly in the Deluge script. The connection's permissions should be limited to the operations required by SATGUARD.

Table 7.3: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Role-based access permissions	Zoho Creator application	Unauthorised access to restricted application functions
Mandatory fields and validation rules	Forms, particularly Telemetry Log	Incomplete or invalid data entry
Authorised Zoho CRM connection	CRM integration workflow	Exposure of authentication credentials in scripts

Security Control	Where Applied	Threat Mitigated
Controlled access to application reports	Reports and dashboards, according to configured permissions	Unauthorised viewing of application data

7.4 Governance and Administration

The administration of SATGUARD involves managing application users, maintaining access permissions, monitoring workflow execution, and ensuring that satellite, telemetry, alert, and maintenance records remain consistent.

The administrator is responsible for maintaining satellite registry information, reviewing application reports, checking alert records, and managing application configuration. Monitoring users are responsible for recording telemetry readings and reviewing generated alerts, while maintenance users handle assigned maintenance activities according to their configured permissions.

The Zoho CRM connection must be monitored to ensure that the integration remains authorised and that the configured case-creation action works correctly. If an integration fails, the workflow configuration, connection permissions, required field mappings, and execution results should be checked.

Changes to forms, Deluge scripts, workflows, reports, and permissions should be tested before they are published to the production environment. A record of significant changes should be maintained to support troubleshooting and future development.

Regular checks should also be performed to identify unresolved alerts, incomplete maintenance tasks, invalid telemetry records, and workflows that are not executing as expected. Backups, audit-history features, and administrative monitoring should be used where supported by the configured Zoho Creator edition.

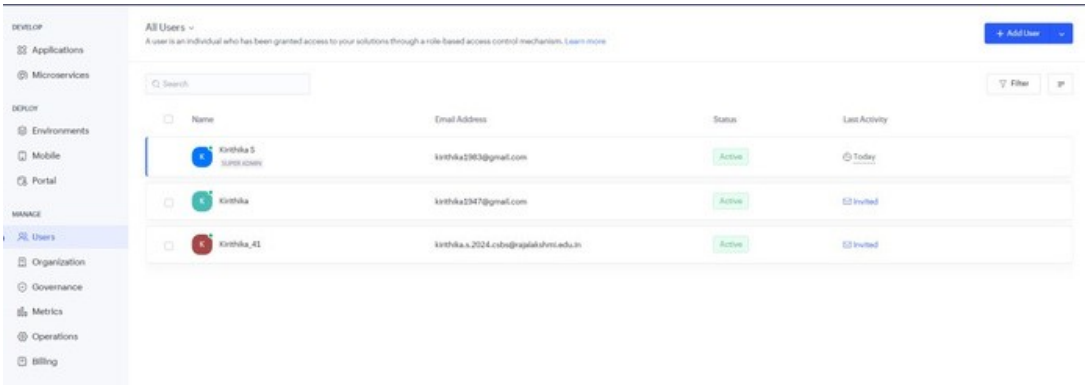


Figure 7.2: Administration Settings

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The SATGUARD application was developed using Zoho Creator to support satellite registry management, telemetry monitoring, automated alert generation, maintenance tracking, and dashboard-based reporting. The application combines forms, reports, Deluge scripts, workflows, and role-based permissions to support the monitoring process.

The following scenario demonstrates the intended workflow of SATGUARD, from registering a satellite to recording telemetry readings and generating alerts when configured conditions are violated.

Step 1: Satellite Registration

A satellite is registered using the Satellite Registry form. The user enters the satellite name, mission type, launch date, orbit type, operational status, and mission description. After the mandatory fields are completed, the form is submitted, and the satellite record becomes available for selection in the Telemetry Log.

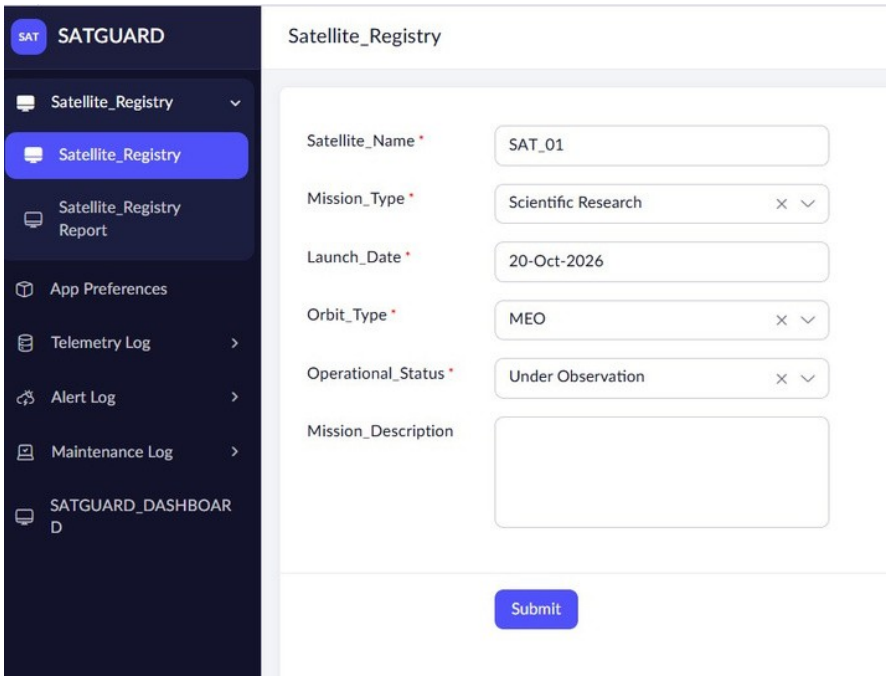


Figure 8.1: Scenario Step 1

Step 2: Telemetry Recording and Alert Generation

Monitoring staff open the Telemetry Log form and select the registered satellite. They enter the battery voltage, temperature, power level, communication status, and reading timestamp.

When the telemetry record is submitted, the configured Deluge workflows evaluate the readings. For example, a low battery voltage, high temperature, low power level, or offline communication status can trigger an alert according to the configured thresholds.

The power-level validation also prevents values below 0 or above 100 from being submitted.

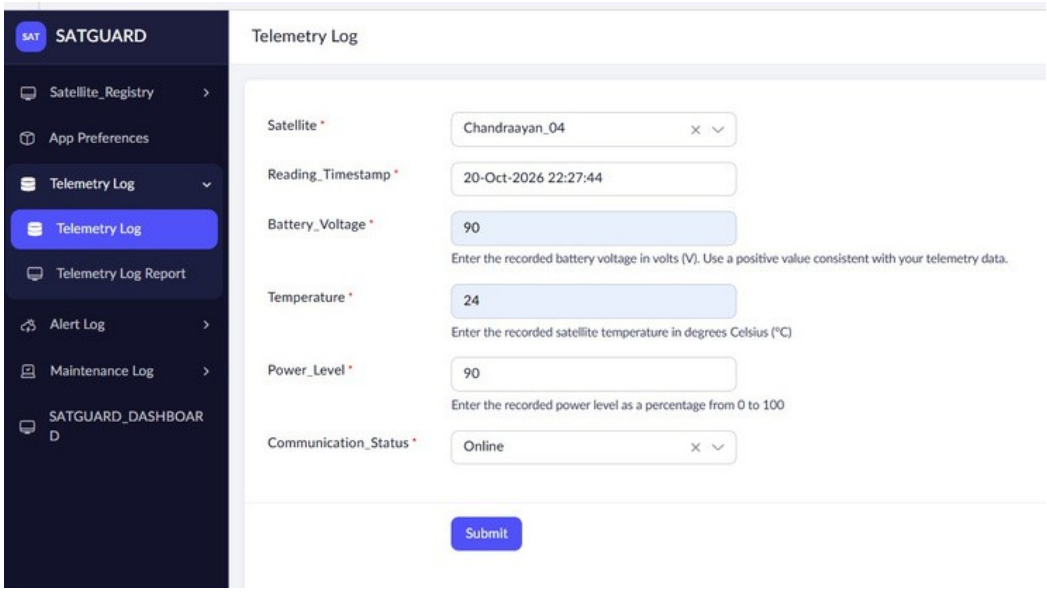


Figure 8.2: Scenario Step 2

Step 3: Alert Review and Maintenance Tracking

The generated alerts are recorded in the Alert Log with details such as the satellite, alert timestamp, alert type, severity, description, and resolution status. Users with the appropriate permissions can review the alert and determine whether maintenance action is required.

Maintenance activities are recorded in the Maintenance Log, which includes the linked alert, maintenance task, assigned person, start date, maintenance status, action taken, and resolution date. The dashboard and reports provide an overview of satellite status, open alerts, and maintenance activities.

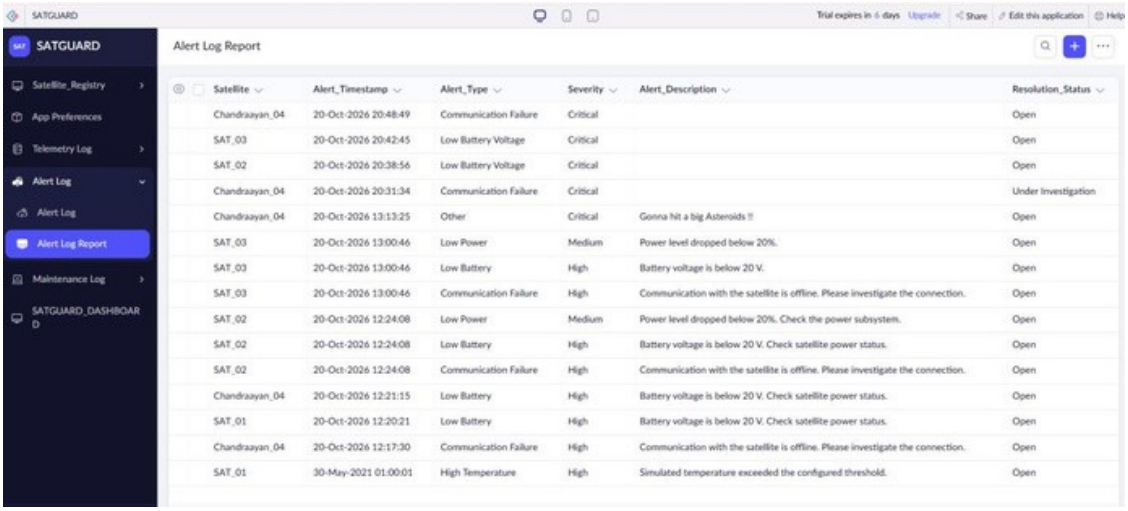


Figure 8.3: Scenario Step 3

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
Develop a centralised satellite information management system.	Satellite Registry form and reports	Satellite details can be stored and accessed in a structured format.
Automate telemetry monitoring and alert generation.	Telemetry Log, Deluge scripts, and alert workflows	Configured telemetry conditions can trigger alerts automatically.
Provide efficient alert management and monitoring.	Alert Log, Maintenance Log, reports, dashboard, and role-based permissions	Users can review alerts, maintain task records, and monitor application information according to their access permissions.

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Time to process one request	Manual checking and recording may require multiple steps.	Form submissions and configured workflows automate selected operations.	Reduced manual effort for supported tasks.
Manual steps	Telemetry readings and alerts may require manual review and recording.	Deluge workflows generate alerts when configured conditions are met.	Less repetitive monitoring work.

Parameter	Existing System	Proposed Application	Improvement
Report generation	Information may need to be compiled from separate records.	Zoho Creator reports and dashboards present organised application data.	Easier access to summaries and records.
Data accessibility	Information may be distributed across separate files or records.	Satellite, telemetry, alert, and maintenance data are managed within SATGUARD.	More centralised information access, controlled by permissions.

8.4 Limitations of the Low-Code Approach

SATGUARD was developed using Zoho Creator, which provides built-in forms, reports, dashboards, workflows, access controls, and Deluge scripting. Although these capabilities simplify development, certain limitations remain.

1. **Platform customisation:** The application's interface and behaviour are influenced by Zoho Creator's available components. Highly specialised interface designs or complex interactions may require additional customisation.
2. **Deluge scripting constraints:** Deluge supports workflow automation and data processing, but complex algorithms and advanced application logic may be more difficult to implement than in a conventional programming environment.
3. **External API integration:** Integrations require valid connections, appropriate permissions, correct field mappings, and compatible API operations. For SATGUARD, the Zoho CRM integration must be tested to confirm that alerts create the intended CRM records successfully.
4. **Platform dependency:** SATGUARD depends on Zoho Creator's availability, supported features, and subscription limits. Moving the application to another platform may require redesigning forms, workflows, and integrations.
5. **Advanced satellite analytics:** The current application focuses on structured telemetry entry, configured threshold checks, alert management, and reporting. Advanced capabilities such as live satellite telemetry ingestion, predictive anomaly detection, and orbital collision analysis would require additional data sources, algorithms, and integrations.

6. **Testing and scalability:** The application requires further testing with larger datasets and multiple users to assess performance under increased workloads.

These limitations can be addressed incrementally through workflow optimisation, additional integrations, improved testing, and custom-code extensions where necessary.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

SATGUARD demonstrates how no-code, low-code, and pro-code approaches can contribute to application development.

No-code: Built-in form designers, reports, dashboards, and permission configuration reduced the need to write code for basic data capture, visualisation, and access management.

Low-code: Deluge scripts and workflows were used to implement telemetry validation, threshold-based alert generation, and automated actions. The Zoho CRM connection extends the application beyond its internal data management capabilities.

Pro-code: A conventional programming approach could be considered if SATGUARD requires complex anomaly-detection models, advanced orbital calculations, real-time telemetry streaming, specialised visualisations, or greater control over infrastructure and APIs.

The low-code approach was suitable for developing the SATGUARD prototype because it enabled the core forms, reports, workflows, and access controls to be built within one platform. It also allowed automated behaviour to be introduced without developing a complete backend from scratch.

In the future, a hybrid architecture could retain Zoho Creator for administrative workflows while using custom services for advanced analytics, real-time data ingestion, and specialised satellite-monitoring functions. Such an extension would depend on the project's technical requirements, available resources, and integration constraints.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

SATGUARD was developed as a satellite-monitoring application using Zoho Creator, a low-code application development platform. The application provides a structured approach to managing satellite information, recording telemetry readings, generating alerts, and tracking maintenance activities. Forms such as Satellite Registry, Telemetry Log, Alert Log, and Maintenance Log were created to organise the application's data. Deluge scripts and workflows were configured to validate telemetry inputs and generate alerts when specified conditions, such as low battery voltage, high temperature, low power level, or communication failure, are detected.

The application also incorporates reports, dashboard components, role-based permissions, and a Zoho CRM connection to extend its functionality beyond basic data management. SATGUARD demonstrates how low-code development can simplify the creation of monitoring and alert-management applications while reducing the amount of conventional programming required. Further testing and enhancements can improve its integration reliability, monitoring capabilities, and suitability for more advanced satellite-monitoring requirements.

9.2 Future Enhancements

The following enhancements could extend the functionality of SATGUARD:

- **Predictive Analytics and Anomaly Detection:** Introduce AI-based analysis to identify unusual telemetry patterns, predict potential equipment failures, and provide early warnings before configured thresholds are exceeded.
- **Real-Time Satellite Data Integration:** Integrate verified external satellite data sources or telemetry APIs to retrieve updated information automatically instead of relying entirely on manually entered readings.
- **Advanced Zoho CRM Integration:** Improve the integration to create and track CRM cases for critical alerts, maintain relevant alert information, and support case-status monitoring.

- **Advanced Dashboard and Visualisation:** Add interactive telemetry charts, historical trend analysis, satellite comparisons, and more detailed alert summaries to improve monitoring and decision-making.
- **Automated Maintenance Management:** Introduce maintenance scheduling, task assignment, completion tracking, and notifications to support a more organised response to satellite alerts.
- **Scalable Backend Integration:** Use a custom backend or a suitable cloud microservice for computationally intensive operations, advanced analytics, and large-scale telemetry processing when required.

9.3 Learning Outcomes and Career Relevance

The development of SATGUARD provided practical experience in low-code application development using Zoho Creator. It involved designing forms, configuring field properties and validation rules, creating reports and dashboards, implementing Deluge scripts, and automating processes through workflows.

The project also provided experience in role-based access management, debugging, application testing, deployment, and external integration using Zoho CRM. Working through these stages helped develop problem-solving skills and an understanding of how application components work together to support a real-world monitoring requirement.

The skills gained are relevant to career paths such as low-code application developer, business application developer, automation engineer, and solutions consultant. The project also establishes a foundation for future work involving API integration, backend development, data analytics, and AI-assisted monitoring systems. Overall, SATGUARD demonstrates the practical application of low-code technologies to develop a structured, workflow-driven application for satellite information and alert management.

REFERENCES

[1] Zoho Corporation, “Zoho Creator Help Documentation.” [Online]. Available: https://www.zoho.com/creator/help/. [Accessed: Oct. 20, 2026].

[2] Zoho Corporation, “Deluge Help Documentation.” [Online]. Available: https://www.zoho.com/deluge/help/. [Accessed: Oct. 20, 2026].

[3] Zoho Corporation, “Zoho Creator Learning Resources.” [Online]. Available: https://www.zoho.com/creator/resources/. [Accessed: Oct. 20, 2026].

[4] Zoho Corporation, “Zoho CRM API Documentation.” [Online]. Available: https://www.zoho.com/crm/developer/docs/api/v8/. [Accessed: Oct. 20, 2026].

[5] Zoho Corporation, “Zoho Creator Connections.” [Online]. Available: https://www.zoho.com/creator/help/app-settings/connections.html. [Accessed: Oct. 20, 2026].

[6] N. Prinz, C. Rentrop, and M. Huber, “Low-Code Development Platforms – A Literature Review,” in Proc. Americas Conference on Information Systems (AMCIS), 2021.

[7] Gartner, Inc., “Critical Capabilities for Enterprise Low-Code Application Platforms,” Gartner Research, 2024.

[8] W. Rosam and O. Kalnichevski, *Low-Code Development: A Practical Guide*. New Delhi, India: BPB Publications.

[9] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2002.

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1 lists the Zoho Creator features covered by the course syllabus (Units I–V) and states whether each feature has been implemented in SATGUARD, where it can be found in the application, and the report section and figure or code that evidence it. Features marked N were not required for the satellite monitoring scenario and are not part of the current implementation.

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done [Y / N]	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	SATGUARD application with left-menu sections for each form, report and the dashboard	5.1	Fig. 5.1
II	Forms with ≥ 8 distinct field types	N	Seven types used: Single Line, Multi Line, Dropdown, Date, Date-Time, Number and Lookup	5.2	Fig. 5.3
II	Lookup field (relationship between forms)	Y	Telemetry Log and Alert Log (Satellite); Maintenance Log (Alert_Reference)	4.2, 5.2	Fig. 4.2
II	Subform (parent–child data)	N	Not used; relationships are implemented through lookup fields	4.2	–
II	Field validations / layout rules	Y	Telemetry Log – power level range 0–100; mandatory fields on all forms	5.2, 5.5	Code 5.1
II	List report with filter, grouping, custom action	N	Four plain list reports implemented; filter, grouping and custom actions not configured	5.3	Fig. 5.8
II	Kanban / Calendar / Timeline / Map report	N	Not implemented	–	–
II	Pivot table / chart report	Y	Donut and bar chart components on SATGUARD_DASHBOARD	5.4	Fig. 5.10

Unit	Zoho Creator Feature	Done [Y / N]	Where in the Application	Report Section	Figure / Code No.
II	Page / dashboard built with UI Builder	Y	SATGUARD_DASHBOARD page with counters, charts and Alert Log report	5.4	Fig. 5.10
II	Deluge – form events (validate / success / field actions)	Y	Telemetry Log – On Validate and On Success scripts	5.5	Code 5.1, Code 5.3
II	Deluge – functions, fetch, loops, collections	Y	Fetch records with criteria for duplicate check; Map collection in CRM script	5.5, 5.7	Code 5.3, Code 5.6
II	Email / in-app notifications	Y	Alert Log – Critical_Alert_Notification (send mail when Severity is Critical)	5.6, 6.3	Fig. 5.11, Fig. 6.1
II	Form workflow	Y	Telemetry Log, Alert Log and Maintenance Log form workflows	5.6	Fig. 5.12
II	Scheduled workflow	N	Not implemented	–	–
II	Approval process	N	Not implemented	–	–
II	Blueprint	N	Not implemented	–	–
II	Batch / report workflow	Y	Batch / report workflow configured in the application	5.6	Fig. 5.12
II	Testing & debugging evidence	Y	Test cases, debugging techniques and defect log	6	Fig. 6.1
III	Integration with a Zoho service	Y	Alert Log → Zoho CRM Cases module (Case creation to be verified, see BUG-03)	5.7	Code 5.6, Fig. 5.13
III	Connection / REST API call (invokeurl)	Y	Authorised Zoho CRM connection used by the integration script; invokeUrl not used	5.7	Fig. 5.13
III	Webhook or Creator REST API	N	Not implemented	–	–
III	Users, roles and permission profiles	Y	Roles and permission profiles for Monitoring Staff and Maintenance Staff	4.6, 5.8	Fig. 5.14, Fig. 5.15
III	Record-level access criteria / customer portal	N	Not implemented; no separate customer portal	5.11	–

Unit	Zoho Creator Feature	Done [Y / N]	Where in the Application	Report Section	Figure / Code No.
III	Zia / AI feature	N	Not used; monitoring is rule-based	5.9	–
III	Analytics (charts / Zoho Analytics)	Y	Dashboard charts and summary counters	5.9	Fig. 5.16
IV	Performance optimisation applied & measured	Y	Input validation, automated checks and independent condition checks; results in Table 7.2	7.2	Table 7.2
IV	Deployment across environments	Y	Development → Stage → Production	7.1	Fig. 7.1
IV	Security controls (encryption, OAuth, permissions)	Y	Role-based permissions, validation rules and authorised CRM connection	7.3	–
IV	Governance (audit trail, administration)	Y	User administration and governance practices	7.4	Fig. 7.2
V	Pro-code extension (Catalyst / widgets / custom functions)	N	Not required for the current design	5.10	–
V	Low-code limitations & spectrum analysis	Y	Limitations and No-code → Low-code → Pro-code reflection	8.4, 8.5	–
—	Mobile app / portal access	Y	Published URL opened in a mobile browser	5.11	Fig. 5.17

Total features implemented: 21 / 32

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1 shows where each Course Outcome of CB23F35 is evidenced in this report and application, with the self-assessed attainment level (1 – Low, 2 – Medium, 3 – High). The examiner level is left blank for evaluation.

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2	3	
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6	3	
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9	2	
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7	2	
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3	2	

Syllabus Coverage

Table B.2 maps each unit of the CB23F35 syllabus to the portion of SATGUARD and of this report in which it is applied.

Table B.2: Mapping of Syllabus Units to the Project

Unit	Syllabus Topics	Applied in SATGUARD	Report Section
I	Fundamentals of low-code application development; overview of Zoho Creator; setting up Zoho Creator; emerging trends in low-code	Justification for a low-code approach, comparison with traditional development, platform setup, emerging trends	1.5, 2.3, 2.5, 5.1
II	Data modelling and forms; reports and dashboards; pages (UI Builder); Deluge scripting; workflows and automation; testing and debugging	Four forms with lookups, list reports, dashboard, Deluge validation and alert scripts, form workflows, test cases and defect log	4.2–4.4, 5.2–5.6, Ch. 6
III	Integration capabilities; user management and role-based access; AI and analytics in Zoho Creator	Zoho CRM connection and Case creation, roles and permission profiles, rule-based analytics on the dashboard	4.6, 5.7–5.9
IV	Performance optimisation; deployment strategies; security; governance and administration	Development–Stage–Production deployment, validation-based optimisation, security controls, administration practices	Ch. 7
V	Limits of low-code; Zoho Creator and Catalyst; no-code to pro-code spectrum; career pathways; future of low-code	Limitations analysis, spectrum reflection, pro-code discussion, future enhancements, career relevance	5.10, 8.4, 8.5, 9.2, 9.3

Laboratory Experiments

Table B.3 maps the fifteen laboratory experiments listed in the syllabus to the work carried out in this project.

Table B.3: Mapping of Laboratory Experiments to the Project

Exp.	Experiment	Evidence in the Project	Status
1	Comparative study of low-code versus traditional development	Table 2.1 and Section 2.3	Completed
2	Setting up a Zoho Creator account and exploring the platform architecture and interface	Sections 4.1 and 5.1	Completed
3	Survey report on emerging trends in low-code development	Sections 2.4 and 2.5	Completed
4	Data modelling and form design for a business application	Sections 4.2, 4.3 and 5.2	Completed
5	Building reports, dashboards and custom pages (UI Builder)	Sections 5.3 and 5.4	Completed

Exp.	Experiment	Evidence in the Project	Status
6	Automating business processes using Deluge scripting and workflows, with testing and debugging	Sections 5.5, 5.6 and Chapter 6	Completed
7	Integrating a Zoho Creator application with another Zoho service using connections and APIs	Section 5.7 (Zoho CRM)	Completed; Case creation to be verified
8	Configuring user management and role-based access control (RBAC)	Sections 4.6 and 5.8	Completed
9	Exploring AI-assisted features and built-in analytics (Zia)	Section 5.9 (rule-based analytics; Zia not used)	Partially covered
10	Applying performance optimisation techniques (indexing, subforms, query tuning)	Section 7.2 (validation and independent condition checks)	Partially covered
11	Deploying the application from development to production	Section 7.1	Completed
12	Implementing security features and governance / administration practices	Sections 7.3 and 7.4	Completed
13	Case study on the limitations of low-code for a complex enterprise use case	Section 8.4	Completed
14	Exploring pro-code extension of a Zoho Creator application using Zoho Catalyst / custom functions	Section 5.10 (discussed; not implemented)	Discussed only
15	Report and presentation on career pathways and the future of low-code	Sections 8.5, 9.2 and 9.3	Completed

CO–PO–PSO Correlation

Table B.4 reproduces the CO–PO–PSO matrix of the course. Correlation levels are defined as 1 – Slight (Low), 2 – Moderate (Medium) and 3 – Substantial (High); "–" indicates no correlation.

Table B.4: CO–PO–PSO Matrix of the Course CB23F35

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
CB23F35.1	3	2	1	1	1	-	-	-	-	1	2	3	2	1	2
CB23F35.2	2	2	3	2	3	-	-	-	2	2	2	2	2	3	3
CB23F35.3	2	3	2	2	3	-	-	-	2	1	2	2	2	2	2
CB23F35.4	1	1	2	1	2	-	-	-	2	2	3	2	1	2	2
CB23F35.5	2	3	2	3	3	-	-	-	1	2	3	3	2	3	2

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
Average	2.0	2.2	2.0	1.8	2.4	-	-	-	1.8	1.6	2.4	2.4	1.8	2.2	2.2

APPENDIX C – COMPLETE DELUGE SOURCE CODE

This appendix lists the Deluge scripts used in SATGUARD, grouped by form, with the Script ID (Table 5.2), the location or trigger, and the purpose of each script. The threshold values are demonstration settings for this project and are not universal spacecraft limits.

C.1 Telemetry Log – On Validate

```
// Script ID : DS-01
// Location  : Telemetry Log -> Form event -> On Validate
// Purpose   : Reject power-level values outside the range 0 to 100

if(input.Power_Level < 0 || input.Power_Level > 100)
{
  alert "Power Level must be between 0 and 100.";
  cancel submit;
}
```

C.2 Telemetry Log – On Success

```
// Script ID : DS-02, DS-03, DS-04
// Location  : Telemetry Log -> Form workflow -> On Success
// Purpose   : Create communication-failure, low-battery,
//             high-temperature and low-power alerts; skip if an
//             open alert of the same type already exists.

// 1. Check communication failure (DS-02)
if(input.Communication_Status == "Offline")
{
  existing = Alert_Log[Satellite == input.Satellite
    && Alert_Type == "Communication Failure"
    && Resolution_Status == "Open"];
  if(existing.count() == 0)
  {
    insert into Alert_Log
    [
      Satellite=input.Satellite
      Alert_Timestamp=zoho.currenttime
      Alert_Type="Communication Failure"
      Severity="High"
      Alert_Description="Communication with the satellite is offline.
          Please investigate."
      Resolution_Status="Open"
      Added_User=zoho.loginuser
    ]
  }
}

// 2. Check low battery (DS-03)
if(input.Battery_Voltage < 20)
{
  existing = Alert_Log[Satellite == input.Satellite
    && Alert_Type == "Low Battery"
```

```
    && Resolution_Status == "Open"];
  if(existing.count() == 0)
  {
    insert into Alert_Log
    [
      Satellite=input.Satellite
      Alert_Timestamp=zoho.currenttime
      Alert_Type="Low Battery"
      Severity="High"
      Alert_Description="Battery voltage is below 20 V."
      Resolution_Status="Open"
      Added_User=zoho.loginuser
    ]
  }
}

// 3. Check high temperature (DS-03)
if(input.Temperature > 40)
{
  existing = Alert_Log[Satellite == input.Satellite
    && Alert_Type == "High Temperature"
    && Resolution_Status == "Open"];
  if(existing.count() == 0)
  {
    insert into Alert_Log
    [
      Satellite=input.Satellite
      Alert_Timestamp=zoho.currenttime
      Alert_Type="High Temperature"
      Severity="High"
      Alert_Description="Satellite temperature exceeded the
          configured threshold."
      Resolution_Status="Open"
      Added_User=zoho.loginuser
    ]
  }
}

// 4. Check low power (DS-03)
if(input.Power_Level < 20)
{
  existing = Alert_Log[Satellite == input.Satellite
    && Alert_Type == "Low Power"
    && Resolution_Status == "Open"];
  if(existing.count() == 0)
  {
    insert into Alert_Log
    [
      Satellite=input.Satellite
      Alert_Timestamp=zoho.currenttime
      Alert_Type="Low Power"
      Severity="Medium"
      Alert_Description="Power level dropped below 20%."
      Resolution_Status="Open"
      Added_User=zoho.loginuser
    ]
  }
```

```
}
```

C.3 Maintenance Log – Completion Date

```
// Script ID : DS-05
// Location  : Maintenance Log -> Form workflow (configured event)
// Purpose   : Record the completion date when a task is marked
    Completed
//             and no resolution date has been entered

if(input.Maintenance_Status == "Completed" && input.Resolution_Date
    == null)
{
  input.Resolution_Date = zoho.currentdate;
}
```

C.4 Alert Log – Zoho CRM Case Creation

```
// Script ID : DS-06
// Location  : Alert Log -> Form workflow (configured event)
// Purpose   : Create a Case in Zoho CRM from an alert

caseDetails = Map();

caseDetails.put("Subject", "SATGUARD Alert - " + input.Alert_Type);
caseDetails.put("Case_Origin", "Web");
caseDetails.put("Status", "New");

descriptionText = "Satellite: " + input.Satellite
  + "\nAlert Type: " + input.Alert_Type
  + "\nSeverity: " + input.Severity
  + "\nAlert Timestamp: " + input.Alert_Timestamp
  + "\nResolution Status: " + input.Resolution_Status
  + "\nDescription: " + input.Alert_Description;

caseDetails.put("Description", descriptionText);

// Use the connection link name from the Zoho Creator connection
    settings
// (not the full connection URL - see defect BUG-03).
response = zoho.crm.createRecord(
  "Cases",
  caseDetails,
  Map(),
  "<crm_connection_link_name>"
);

info response;
```

The Critical_Alert_Notification workflow is configured without script: Alert Log → Created → Successful form submission → execute only if Severity == "Critical" → Send mail.

APPENDIX D – USER MANUAL

D.1 Accessing the Application

- **Web:** Open a supported browser (Google Chrome, Microsoft Edge or Mozilla Firefox) and enter the published application URL given in the Project at a Glance table.
- Sign in with the Zoho account assigned to your role. Only the menu items permitted for the role are displayed in the left-hand menu.
- **Mobile:** Open the same URL in a mobile browser or sign in through the Zoho Creator mobile app. The dashboard adapts to the screen as shown in Figure 5.17.
- Use **SATGUARD_DASHBOARD** from the left-hand menu to view the satellite, alert and maintenance summary.

D.2 Instructions for the Administrator

1. Open **Satellite_Registry → Satellite_Registry**, enter Satellite_Name, Mission_Type, Launch_Date, Orbit_Type, Operational_Status and (optionally) Mission_Description, and click **Submit**.
2. Open **Satellite_Registry Report** to view, search, edit or delete satellite records.
3. Open **SATGUARD_DASHBOARD** to review the number of satellites, operational satellites, open alerts and maintenance tasks, together with the charts.
4. Open **Alert Log → Alert Log Report** to review alerts; when an issue is resolved, edit the record and change Resolution_Status.
5. Add users and assign the Monitoring Staff or Maintenance Staff profile from the application **Settings → Users** section.
6. Periodically confirm that the workflows are enabled and that the Zoho CRM connection is authorised.

D.3 Instructions for Monitoring Staff

1. Open **Telemetry Log → Telemetry Log**.
2. Select the Satellite, enter Reading_Timestamp, Battery_Voltage (V), Temperature (°C), Power_Level (0–100) and Communication_Status, and click **Submit**.

3. If Power_Level is outside 0–100, the message “Power Level must be between 0 and 100.” is displayed; correct the value and submit again.
4. After submission, the application checks the readings: an Offline status, battery voltage below 20 V, temperature above 40 and power level below 20 each create an alert unless an open alert of the same type already exists for the satellite.
5. Open **Alert Log → Alert Log Report** to review alert type, severity and resolution status. Critical alerts also trigger an email notification.
6. Use the Satellite Registry Report and the dashboard (view only) to check satellite status.

D.4 Instructions for Maintenance Staff

1. Open **Alert Log → Alert Log Report** (view only) to identify alerts that require maintenance action.
2. Open **Maintenance Log → Maintenance Log**, select the Alert_Reference, enter Maintenance_Task, Assigned_To, Start_Date and Maintenance_Status, and click **Submit**.
3. To update a task, open **Maintenance Log Report**, edit the record, enter the Action_Taken and set Maintenance_Status to Completed. If Resolution_Date is empty, the system fills in the current date automatically.
4. Use the dashboard (view only) to check the number of open maintenance tasks.

D.5 Common Messages

Table D.1: Common Messages and Actions

Message / Situation	Meaning	Action
Power Level must be between 0 and 100.	The Power_Level value is outside the permitted range.	Enter a value from 0 to 100 and submit again.
Required field highlighted on a form	A mandatory field has been left empty.	Complete all fields marked with *.
A menu item or report is not visible	The role does not have permission for that component.	Contact the Administrator.

Message / Situation	Meaning	Action
No new alert after a submission	An open alert of the same type already exists for the satellite.	Review the existing alert in the Alert Log Report.